

RTL Design Sherpa

Lattice Micro-Architecture Specification 1.2

October 13, 2026

Table of Contents

Lattice Mas Index....16

Lattice Micro-Architecture Specification Index....17

Document Organization....18

Main Documentation....18

Chapter 1: Architecture....19

Chapter 2: Address Decode and Arbitration....20

Chapter 3: RTL Generator....21

Quick Navigation....22

For New Users....22

For Integration....22

Common Questions....22

Visual Assets....23

Architecture Diagrams....23

Timing Diagrams....23

Component Overview....24

Key Features....24

Pre-Generated Variants....24

Design Philosophy....24

Related Documentation....25

Companion Specifications....25

Project-Level....25

Test Infrastructure....25

RTL 25
Version History 26
Lattice Visual Documentation 27
Overview 28
Documentation Structure 29
Main Documentation Files 29
Supporting Documentation 29
Visual Assets 30
Architecture Diagrams 30
Timing Diagrams 30
Quick Start 31
For New Users 31
For Existing Users 31
Directory Structure 32
Key Concepts 33
1. Architecture 33
2. Address Decode 33
3. Arbitration 33
4. RTL Generation 33
Common Questions 35
Q: Which pre-generated variant should I use? 35
Q: How do I change the base address? 35
Q: Can I change per-slave address sizes? 35
Q: How does arbitration work with 3+ masters? 35

Q: What's the throughput?..... 36
Regenerating Diagrams..... 37
Graphviz Diagrams (Architecture, Address Decode)..... 37
WaveJSON Diagrams (Timing)..... 37
Version History..... 38
Maintainers..... 39
Lattice Architecture..... 40
Overview..... 41
Architecture Diagram..... 42
Figure 1.1: Lattice Top-Level Architecture..... 42
Functional Blocks..... 43
1. Master-Side Protocol Conversion..... 43
2. Internal Crossbar Logic..... 43
3. Slave-Side Protocol Conversion..... 43
Signal Flow Example..... 45
Parameter Configuration..... 46
APB5 Parity Across the Fabric..... 47
APB5 PWAKEUP Across the Fabric..... 48
Pre-Generated Variants..... 49
Design Philosophy..... 50
Next Steps..... 51
Address Decode and Arbitration..... 52
Overview..... 53
Address Decode..... 54

Address Map Structure....54
Decode Algorithm....54
Out-of-Range Addresses....54
Address Decode Flow Diagram....55
Figure 2.1: Address Decode Flow....55
Decode Example Walkthrough....55
Multiple Address Maps....56
Arbitration....57
Per-Slave Round-Robin....57
Round-Robin Timing Diagram....57
Waveform 2.1: Round-Robin Arbitration Timing....57
Arbitration Example Walkthrough....57
Multi-Slave Parallelism....58
Grant Persistence....59
Arbitration Latency....59
Integration Examples....60
Example 1: CPU to 4 Peripherals (No Contention)....60
Example 2: CPU + DMA to 4 Peripherals (Potential Contention)....60
Performance Characteristics....61
Throughput....61
Latency....61
Design Notes....62
Why 64KB Per Slave?....62
Why Round-Robin?....62

Troubleshooting..... 63
Issue: Wrong Slave Selected..... 63
Issue: Master Starved..... 63
Issue: Back-to-Back Transactions Stalling..... 63
Next Steps..... 64
Lattice RTL Generator..... 65
Overview..... 66
Generator Architecture..... 67
Two-Level Structure..... 67
Quick Start..... 68
Generate Standard Variants (1:1, 2:1, 1:4, 2:4)..... 68
Generate Custom Variant..... 68
Generate with Custom Base Address..... 68
Command-Line Interface..... 70
Full Options..... 70
Examples..... 71
Generated Code Structure..... 72
Template Components..... 72
Generator Algorithm..... 76
Step-by-Step Process..... 76
Code Generation Patterns..... 78
Loop-Based Generation..... 78
String Formatting..... 78
Customization Examples..... 79

Example 1: Large SoC with 10 Peripherals....79
Example 2: Multi-CPU System....79
Example 3: Memory-Mapped I/O Region....79
Using Generated Modules....81
Integration Example....81
Parameter Override....82
Generator Limitations and Future Work....83
Current Limitations....83
Future Enhancements....83
Testing Generated Code....85
Verification Flow....85
Recommended Test Cases....85
Advanced Usage....86
Using as Python Library....86
Batch Generation....86
Troubleshooting....87
Issue: “ModuleNotFoundError: No module named ‘amba.apbx_xbar_generator’”....87
Issue: Generated Code Won’t Lint....87
Issue: Generated Code Too Large....87
References....88
Verification Wrappers....89
Lattice Hardware Architecture Specification Index....90
Document Organization....91

Front Matter..... 91
Chapter 1: Introduction..... 91
Chapter 2: System Overview..... 91
Chapter 3: Architecture..... 91
Chapter 4: Interfaces..... 91
Chapter 5: Performance..... 91
Chapter 6: Integration..... 91
Related Documentation..... 93
Product Requirements Document (PRD)..... 94
Lattice Generator..... 94
1. Executive Summary..... 95
1.1 Quick Stats..... 95
1.2 Project Goals..... 95
2. Key Design Principles..... 96
2.1 Architecture Philosophy..... 96
2.2 Design Trade-offs..... 96
3. Architecture Overview..... 97
3.1 Top-Level Block Diagram..... 97
3.2 Address Mapping..... 98
3.3 Arbitration Strategy..... 99
4. Available Variants..... 101
4.1 Pre-Generated Modules..... 101
4.2 Wrapper Modules..... 101
5. Functional Requirements..... 103

FR-1: Arbitrary MxN Configuration....103
FR-2: Round-Robin Arbitration....103
FR-3: Address-Based Routing....103
FR-4: Back-to-Back Transactions....103
FR-5: Configurable Address Map....103
6. Interface Specifications....104
6.1 Master-Side Interface (APB Slave)....104
6.2 Slave-Side Interface (APB Master)....104
7. Parameter Configuration....105
8. Generator Usage....106
8.1 Pre-Generated Variants....106
8.2 Custom Generation....106
8.3 Generator Options....106
9. Performance Characteristics....107
9.1 Latency....107
9.2 Throughput....107
9.3 Resource Utilization (Estimated)....107
10. Verification Status....108
10.1 Test Coverage....108
10.2 Test Methodology....108
11. Integration Guide....109
11.1 Simple Integration (1to4)....109
11.2 Multi-Master Integration (2to4)....109
12. Design Constraints....111

12.1 Limitations....111
12.2 Assumptions....111
13. Future Enhancements....112
13.1 Potential Features....112
13.2 Generator Improvements....112
14. References....113
14.1 Internal Documentation....113
14.2 External Standards....113
14.3 Related Components....113
Claude Code Guide: Lattice Component....114
Quick Context....115
Critical Rules for This Component....116
Rule #0: This is a GENERATOR, Not Just Modules....116
Rule #1: Address Map is Fixed Per-Slave....116
Rule #2: Know the Pre-Generated Variants....116
Rule #3: Generator Syntax....117
Architecture Quick Reference....118
Module Organization....118
Block Diagram (2×4 Example)....118
Common User Questions and Responses....120
Q: “How do I connect a CPU to 4 peripherals?”....120
Q: “I need CPU and DMA to access peripherals. Which crossbar?”....120
Q: “What if I need 3 masters and 8 slaves?”....121
Q: “How does arbitration work?”....121

Q: “Can I change the address map?” 122
Q: “How do I run tests?” 122
Integration Patterns 124
Pattern 1: Simple Peripheral Interconnect 124
Pattern 2: Multi-Master System 124
Pattern 3: Hierarchical Interconnect 125
Anti-Patterns to Catch 126
Anti-Pattern 1: Generating When Pre-Generated Exists 126
Anti-Pattern 2: Assuming Custom Per-Slave Sizes 126
Anti-Pattern 3: Not Mentioning Address Map 126
Anti-Pattern 4: Forgetting About Wrappers 126
Debugging Workflow 128
Issue: Address Not Routing Correctly 128
Issue: Arbitration Not Fair 128
Issue: Back-to-Back Transactions Stalling 128
Quick Commands 129
Remember 130
Lattice — APB Crossbar 131
Architecture 132
Key Features 132
Why mixing needs no converters 132
Available Modules 133
Generating Crossbars 134
Method 1: Convenience Script (Recommended) 134

Method 2: Direct Generator....134
Address Map....135
Parameters....136
Usage Example....137
Testing....138
Design Notes....139
Arbitration Strategy....139
Address Decoding....139
Timing....139
Known Limitations....140
Generating Custom Variants....141
Files....142
References....143
Not implemented....144

List of Figures

Figure 1.1: Lattice Top-Level Architecture..... 42
Figure 2.1: Address Decode Flow..... 55

List of Tables

Table 1: Generated-Variant Parameters.....46
Table 2: Command Line Options.....69
Table 3: Python API Arguments.....69
Table 4: Verification Wrapper Parameters.....88

List of Waveforms

Waveform 2.1: Round-Robin Arbitration Timing.....57

Lattice Mas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Lattice Micro-Architecture Specification Index

Component: Lattice (MxN Interconnect) **Version:** 1.2 **Date:** 2026-10-12 **Purpose:** Detailed micro-architecture specification for Lattice component

---

Document Organization

This specification covers the Lattice: a parametric MxN interconnect that routes multiple APB masters to multiple APB slaves with automatic address-based routing and round-robin arbitration.

Main Documentation

README.md

Chapter 1: Architecture

chapters/01_architecture.md

Chapter 2: Address Decode and Arbitration

chapters/02_address_and_arbitration.md

Chapter 3: RTL Generator

chapters/03_rtl_generator.md

Quick Navigation

For New Users

1. Start with README.md for overview and quick start
2. Read 01_architecture.md to understand the design
3. Study 02_address_and_arbitration.md for operational details
4. Reference 03_rtl_generator.md if you need a custom configuration

For Integration

- **Pre-generated variants:** See 01_architecture.md Section “Pre-Generated Variants”
- **Custom generation:** See 03_rtl_generator.md Section “Quick Start”
- **Address mapping:** See 02_address_and_arbitration.md Section “Address Decode”
- **Arbitration behavior:** See 02_address_and_arbitration.md Section “Arbitration”

Common Questions

All answered in README.md, section “Common Questions”.

Visual Assets

Every diagram in this specification is checked in as both source and rendered output:

- **Source Files:**
  - `assets/graphviz/*.dot` - Graphviz source diagrams
  - `assets/wavedrom/*.json` - WaveJSON timing diagrams
- **Rendered Files:**
  - `assets/graphviz/*.png` - PNG format (for document generation)

Architecture Diagrams

1. **Lattice Architecture (2x4 Example)**
   - Source: assets/graphviz/apbx_xbar_architecture.dot
   - Rendered: assets/graphviz/apbx_xbar_architecture.png
2. **Address Decode Flow**
   - Source: assets/graphviz/address_decode_flow.dot
   - Rendered: assets/graphviz/address_decode_flow.png

Timing Diagrams

1. **Round-Robin Arbitration**
   - Source: assets/wavedrom/arbitration_round_robin.json
   - Rendered: assets/wavedrom/arbitration_round_robin.png

Component Overview

Key Features

- **Parametric MxN Configuration:** Any combination of M masters and N slaves (up to 16x16)
- **Automatic Address Decode:** 64KB per slave, simple offset-based routing
- **Round-Robin Arbitration:** Per-slave fair arbitration, no master starvation
- **Back-to-Back Transactions:** Accepted without master-side idle cycles, though they do not overlap inside the fabric (10 pclk cycles each for M = 1, 11 for arbitrated variants; one more than that variant's single-transfer latency — see HAS 5.2)
- **Grant Persistence:** Grant held through transaction completion
- **RTL Generation:** Python-based generator for custom configurations

Pre-Generated Variants

Module	M×N	Use Case
apbx_xbar_1to1	1×1	Passthrough, protocol conversion
apbx_xbar_2to1	2×1	Multi-master arbitration
apbx_xbar_1to4	1×4	Simple SoC peripheral bus
apbx_xbar_2to4	2×4	Typical SoC with CPU+DMA
apbx_xbar_2to2_mixed	2×2	Mixed APB4/APB5 (m0=APB4, m1=APB5; s0=APB5, s1=APB4)

Design Philosophy

Proven Building Blocks: - Built from production-tested `apb4_slave` and `apb4_master` modules - No new protocol logic - pure composition - Each component independently verified

Parametric Generation: - Generator creates any MxN configuration - Pre-generated common variants for fast integration - Custom variants generated on-demand

Clean Separation: - Master-side: APB slaves convert protocol → cmd/rsp - Internal: Arbitration + address decoding - Slave-side: APB masters convert cmd/rsp → protocol

---

Related Documentation

Companion Specifications

- **Lattice HAS** - Hardware Architecture Specification (high-level)

Project-Level

- **PRD.md:** ../../PRD.md - Complete product requirements document
- **CLAUDE.md:** ../../CLAUDE.md - AI assistant integration guide
- **README.md:** ../../README.md - Quick start guide

Test Infrastructure

- **Test Directory:** `../../dv/tests/` - CocoTB + pytest test suite
- **Test Results:** 100% passing (9/9) — all pre-generated variants plus the dedicated timing test; `apbx_xbar_2to2_mixed` additionally has a formal harness

RTL

- **Core Modules:** `../../rtl/apbx_xbar_*.sv` - Pre-generated crossbars
- **Wrappers:** `../../rtl/wrappers/` - testbench scaffolds (pclk/presetn only), not integration wrappers
- **Generator:** `../../bin/generate_xbars.py` - Python generator script

Version History

Version 1.0 (2025-10-25): - Initial specification release - Complete visual documentation (3 diagrams) - Generator documentation - All pre-generated variants verified (100% passing)

Version 1.1 (2026-10-12): - Refreshed against current RTL, generator, and DV - Updated verification status (9/9 tests passing, mixed-variant formal harness) - Added `--slave-size` CLI option to generator table - Updated line-of-code counts (single source of truth remains HAS ch05_performance/03_resources.md)

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Lattice Visual Documentation

Component: Lattice (MxN Interconnect) **Version:** 1.2 **Status:** Production Ready **Last Updated:** 2026-10-12

Overview

This directory holds the Lattice's documentation set: the architecture chapters, their diagrams and timing waveforms, and the generator guide.

Component Purpose: Parametric MxN APB interconnect connecting multiple masters to multiple slaves with automatic address-based routing and round-robin arbitration.

Documentation Structure

Main Documentation Files

Document	Description	Contents
01_architecture.md	Architecture Overview	Top-level block diagram, functional blocks, signal flow, parameters
02_address_and_arbitration.md	Address Decode & Arbitration	Address map structure, decode algorithm, round-robin timing
03_rtl_generator.md	RTL Generator Guide	Generator architecture, usage, customization, advanced topics

Supporting Documentation

Document	Location	Description
PRD.md	../../PRD.md	Complete product requirements document
CLAUDE.md	../../CLAUDE.md	AI assistant integration guide
README.md	../../README.md	Quick start guide

Visual Assets

Architecture Diagrams

Diagram	Format	Description
apbx_xbar_architecture	Graphviz	2x4 crossbar showing master-side slaves, arbitration, slave-side masters
address_decode_flow	Graphviz	Step-by-step address decode example (0x10023456 to Slave 2)

Rendered Format: PNG beside each source in `assets/graphviz/` (what the chapters embed).

Timing Diagrams

Diagram	Format	Description
arbitration_round_robin	WaveJSON	2 masters competing for Slave 0 with round-robin arbitration

Rendered Format: - PNG: `assets/wavedrom/arbitration_round_robin.png`

Quick Start

For New Users

1. **Understand the Architecture**
   - Read 01_architecture.md
   - Study the architecture diagram
   - Understand master-side/slave-side protocol conversion
2. **Learn Address Decode & Arbitration**
   - Read 02_address_and_arbitration.md
   - Study address decode flow diagram
   - Study arbitration timing diagram
3. **Generate Custom Crossbar (if needed)**
   - Read 03_rtl_generator.md
   - Use pre-generated variants (1to1, 2to1, 1to4, 2to4, 2to2_mixed) if possible
   - Run generator for custom MxN configurations
4. **Integrate Into Your Design**
   - See integration examples in CLAUDE.md
   - Reference complete specification in PRD.md

For Existing Users

Need a Crossbar? - 1 master, N slaves → Use `apbx_xbar_1toN` or generate - 2 masters, N slaves → Use `apbx_xbar_2toN` or generate - M masters, N slaves → Generate with `generate_xbars.py`

Understanding Behavior? - Address decode issues → See 02_address_and_arbitration.md - Arbitration questions → See 02_address_and_arbitration.md

Modifying/Generating? - Generator usage → See 03_rtl_generator.md

---

Directory Structure

```
docs/lattice_mas/
├── README.md                                ← This file
├── lattice_mas_index.md                    ← Book index (PDF build)
├── chapters/
│   ├── 01_architecture.md                   ← Architecture overview
│   ├── 02_address_and_arbitration.md        ← Decode & arbitration
details
│   └── 03_rtl_generator.md                  ← Generator documentation
└── assets/
    ├── graphviz/                            ← Diagram sources +
rendered PNGs
    │   ├── apbx_xbar_architecture.dot / .png
    │   ├── address_decode_flow.dot / .png
    │   └── regenerate_all_graphviz.sh
    └── wavedrom/                            ← Source timing diagrams
        └── arbitration_round_robin.json
```

Key Concepts

1. Architecture

Three-Stage Design: 1. **Master-Side:** APB slaves convert protocol → cmd/rsp 2. **Internal Logic:** Address decode + arbitration + response routing 3. **Slave-Side:** APB masters convert cmd/rsp → protocol

Why This Design? - Reuses proven `apb4_slave` and `apb4_master` components - Clean separation of concerns - Scalable to any MxN configuration

2. Address Decode

Fixed 64KB Per Slave:

```
Slave 0: BASE_ADDR + 0x00000 - 0x0FFFF
Slave 1: BASE_ADDR + 0x10000 - 0x1FFFF
Slave 2: BASE_ADDR + 0x20000 - 0x2FFFF
...
```

Decode Formula:

```
offset = PADDR - BASE_ADDR
slave_index = offset[16 +: $clog2(S)]  // [17:16] for S=4, [16] for
S=2
```

See: 02_address_and_arbitration.md

3. Arbitration

Per-Slave Round-Robin: - Each slave has independent arbiter - Priority rotates after each grant - Grant persists through transaction completion - No master starvation

See: 02_address_and_arbitration.md

4. RTL Generation

All Generator Code in Component Area: - **Convenience Wrapper:** `bin/generate_xbars.py` (top-level script) - **Core Generator:** `bin/apbx_xbar_generator.py` (library implementation)

Usage:

```
# Generate standard variants
python generate_xbars.py

# Generate custom 3x6 crossbar
python generate_xbars.py --masters 3 --slaves 6
```

See: 03_rtl_generator.md

Common Questions

Q: Which pre-generated variant should I use?

Decision Tree:

```
1 master, 1 slave  → apbx_xbar_1to1 (passthrough)
1 master, 4 slaves → apbx_xbar_1to4 (address decode only)
2 masters, 1 slave → apbx_xbar_2to1 (arbitration only)
2 masters, 4 slaves → apbx_xbar_2to4 (full crossbar)
2 masters, 2 slaves (mixed APB4/APB5) → apbx_xbar_2to2_mixed
Other MxN          → Generate with generate_xbars.py
```

Q: How do I change the base address?

At Instantiation:

```
apbx_xbar_1to4 #(
    .BASE_ADDR(32'h8000_0000)  // Override default 0x10000000
) u_xbar (...);
```

Or Generate with Custom Base:

```
python generate_xbars.py --masters 2 --slaves 4 --base-addr 0x80000000
```

Q: Can I change per-slave address sizes?

No: each slave is fixed at a 64KB window. Only the map's origin is settable, via the `BASE_ADDR` parameter.

Workarounds: 1. Use multiple crossbars with different BASE_ADDR 2. Modify generator's decode logic (advanced) 3. Address translation in slaves

See: 03_rtl_generator.md#limitations

Q: How does arbitration work with 3+ masters?

Round-Robin Example (4 Masters, Same Slave):

```
Initial priority: M0
Transaction 1: M0 requests → M0 granted → Priority rotates to M1
Transaction 2: M0, M1, M2 request → M1 granted → Priority rotates to
M2
Transaction 3: M0, M3 request → M2 skipped (not requesting) → M3
granted → Priority rotates to M0
Transaction 4: M0, M1 request → M0 granted → Priority rotates to M1
```

Key: Priority rotates after each grant, ensuring fairness.

See: 02_address_and_arbitration.md#arbitration

Q: What's the throughput?

Single Master: - Back-to-back transactions without master-side gaps (no overlap inside the fabric; 10 pclk cycles each for M = 1, 11 when arbitrated) - Limited only by slave PREADY response time

Multiple Masters (Same Slave): - Fair sharing: Each master gets ~1/M bandwidth - Example: 2 masters = 50% each, 4 masters = 25% each

Multiple Masters (Different Slaves): - Full parallelism: Each master gets 100% of its target slave - Total system bandwidth = (number of slaves) × slave_bandwidth

See: 02_address_and_arbitration.md#performance

Regenerating Diagrams

Graphviz Diagrams (Architecture, Address Decode)

```
# Renders every .dot beside itself as PNG (150 dpi, alpha flattened,
PNG8)
assets/graphviz/regenerate_all_graphviz.sh
```

WaveJSON Diagrams (Timing)

```
cd assets/wavedrom/

# Generate SVG, then PNG
wavedrom-cli -i arbitration_round_robin.json -s
arbitration_round_robin.svg
wavedrom-cli -i arbitration_round_robin.json -p
arbitration_round_robin.png
```

Version History

Version 1.0 (2025-10-25): - Initial visual documentation release - 3 main documentation files - 2 Graphviz architecture diagrams - 1 WaveJSON timing diagram - RTL generator documentation

Version 1.1 (2026-10-12): - Refreshed against current RTL, generator, and DV - Added mixed-variant and timing-test guidance

---

Maintainers

RTL Design Sherpa Project

Questions or Issues: 1. Check this documentation first 2. Review PRD.md for complete specification 3. Review CLAUDE.md for integration guidance 4. Open a GitHub issue if the problem persists

---

Last Updated: 2026-10-12 **Version:** 1.2 **Status:** Production Ready

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Lattice Architecture

Component: Lattice (MxN Interconnect) **Version:** 1.2 **Status:** Production Ready

Overview

The Lattice is a parametric interconnect that connects M APB masters to N APB slaves with automatic address-based routing and per-slave round-robin arbitration. It is a pure composition of the proven `apb4_slave` and `apb4_master` components — there is no new protocol logic in the crossbar itself.

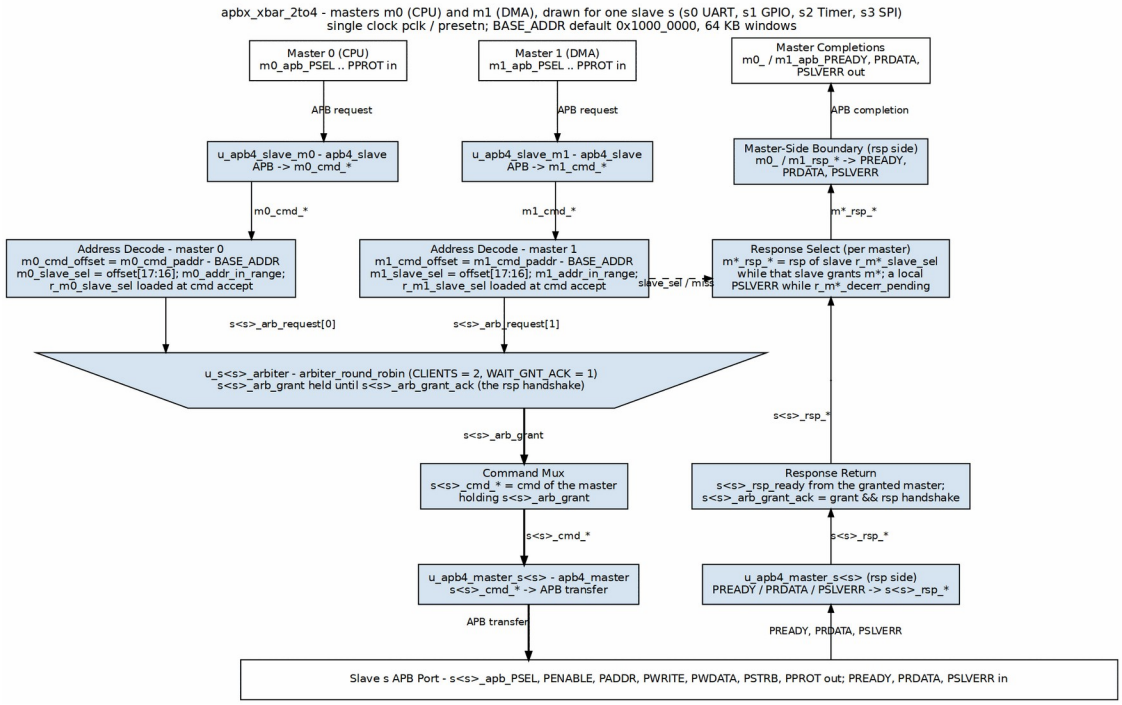
Key Features: - Arbitrary MxN configuration (up to 16x16) - Automatic address decode (64KB per slave) - Round-robin arbitration per slave - Back-to-back transactions (no overlap; 10 pclk cycles each for M = 1, 11 when arbitrated) - Grant persistence through transaction completion

---

Architecture Diagram

The 2x4 configuration below connects 2 masters (CPU and DMA) to 4 slaves (UART, GPIO, Timer, SPI):

Figure 1.1: Lattice Top-Level Architecture



Lattice Architecture (2x4 Example)

Both masters are drawn, for one slave `s`. Each master's cmd is decoded into a request bit for the slave it addresses; the slave's `arbiter_round_robin` (the trapezium) grants one master and holds the grant until the response handshake (`WAIT_GNT_ACK = 1`); the command mux and `apb4_master` carry the granted cmd to the slave. The response climbs the right column, steered back by the slave index registered at cmd accept, or replaced by a local PSLVERR on a decode miss.

Functional Blocks

1. Master-Side Protocol Conversion

Component: `apb4_slave[M]` instances (one per master)

Purpose: Convert incoming APB protocol transactions to internal cmd/rsp bus format

Features: - Full APB protocol handling (PSEL, PENABLE, PREADY) - Transaction buffering - Error response generation - Back-to-back transaction support

Dataflow:

```
APB Master → apb4_slave → cmd/rsp bus → Internal Crossbar Logic
```

---

2. Internal Crossbar Logic

Components: - **Address Decode** - Parallel decode to determine target slave - **Per-Slave Arbiters** - Round-robin arbitration for each slave - **Response Routing** - Registered paths back to requesting masters

Key Operations:

Address Decode:

```
offset = PADDR - BASE_ADDR
slave_index = offset[16 +: $clog2(S)]  // [17:16] for S=4, [16] for
S=2
```

Arbitration: - Independent arbiter per slave - Round-robin priority rotation - Grant persistence through response - No starvation guarantee

Response Routing: - Track which master initiated each transaction - Route PRDATA/PSLVERR back to originating master - Maintain transaction ordering per master

---

3. Slave-Side Protocol Conversion

Component: `apb4_master[N]` instances (one per slave)

Purpose: Convert internal cmd/rsp bus format back to APB protocol for slaves

Features: - APB protocol generation (PSEL, PENABLE timing) - Response collection (PRDATA, PSLVERR, PREADY) - Wait state handling - Error propagation

Dataflow:

```
Internal Crossbar Logic → cmd/rsp bus → apb4_master → APB Slave
```

Signal Flow Example

Transaction: Master 0 (CPU) writes to Slave 2 (Timer) at address 0x10023456

Step-by-step:

1. **Master 0 → apb4_slave[0]:**
   - CPU asserts PSEL, PADDR=0x10023456, PWRITE=1
   - apb4_slave[0] converts to cmd/rsp format
2. **Address Decode:**
   - offset = 0x10023456 - 0x10000000 = 0x00023456
   - slave_index = 0x00023456 >> 16 = 0x2
   - Target: Slave 2
3. **Arbiter[2]:**
   - Check if Slave 2 is available
   - Grant to Master 0 (if no conflict)
   - Route cmd to apb4_master[2]
4. **apb4_master[2] → Slave 2:**
   - Generate APB write transaction
   - Assert PSEL[2], PENABLE, PADDR, PWDATA
   - Wait for PREADY
5. **Response Path:**
   - Slave 2 responds with PREADY, PSLVERR
   - apb4_master[2] captures response
   - Response routed back to apb4_slave[0]
   - apb4_slave[0] returns PREADY to CPU

Total Latency: 9 cycles SETUP-to-PREADY uncontended on a single-master variant, 10 when arbitrated; sustained back-to-back cadence is 10 and 11 respectively (measured; see 2.x — the fabric's boundary IP and registered skid buffers dominate, not APB's 2-cycle protocol minimum)

Parameter Configuration

Every variant has its M×N **baked in** by the generator, so there is no port-count parameter to override at elaboration — a different shape means regenerating, not re-elaborating. What remains parameterizable:

Generated-Variant Parameters

Parameter	Range	Default	Description
ADDR_WIDTH	18-64 (decoding variants); 1-64 (1to1/2to1)	32	Address bus width
DATA_WIDTH	8-64	32	Data bus width
STRB_WIDTH	derived	DATA_WIDTH/8	Write strobe width
BASE_ADDR	any except the top S x 64KB (where BASE+span wraps 32-bit)	0x10000000	Base of the slave address map

Port counts, per-slave window size and APB version per port are generator inputs, not parameters — see chapter 3.

Derived: - Slave address range: 64KB (0x10000) per slave, as the shipped variants are generated (the generator's own default window is 4KB) - Total address space: S × 64KB

APB5 Parity Across the Fabric

Parity is carried only between two APB5 ports. A **mixed pairing ignores parity entirely**, gated by the same `MST_APB5`/`SLV_APB5` masks as the rest of the APB5 sideband — there is no separate policy knob.

The parity pins are the AMBA APB (IHI 0024E) chapter 5 `*CHK` signals: `PADDRCHK`, `PCTRLCHK`, `PSELCHK`, `PENABLECHK`, `PWDATACHK`, `PSTRBCHK`, `PAUSERCHK`, `PWUSERCHK`, `PWAKEUPCHK`, `PRDATACHK`, `PREADYCHK`, `PSLVERRCHK`, `PRUSERCHK` and `PBUSERCHK`. Parity is odd, with one check bit per 8 bits of the covered signal (Table 5-1).

On an APB5→APB5 path the fabric **checks and regenerates**, and its architecture leaves no alternative. The boundary IP deconstructs each transfer into cmd/rsp, and the parity bits do not cross that interface: `apb5_slave` checks on the way in, `apb5_master` regenerates on the way out. The cmd/rsp fabric between them is therefore **outside the protected domain** — corruption *inside* that span is invisible, because the regenerated bit is correct by construction and masks it. That span is documented rather than hidden, and because of it each APB5 port brings its `parity_error_*` flag out individually: a check whose result goes nowhere is not protection.

The error flags are deliberately **not** folded into `PSLVERR`, which would make a fabric fault indistinguishable from the slave's own error response.

None of this is present in the SHIPPED variants. All of the above describes a variant generated with `enable_parity=True`. Every crossbar in `rtl/` is generated with parity OFF: `apbx_xbar_2to2_mixed` declares no parity ports at all on its module boundary, ties the boundary IP's parity inputs to `'0` and leaves its `parity_error_*` outputs unconnected. So there is nothing to connect and nothing to monitor unless you generate your own variant with the flag set — and a mixed APB4/APB5 pairing carries no parity in any case.

Formal coverage: `formal/apbx_xbar/apbx_xbar_2to2_mixed/` proves the mixed configuration as of 2026-08-29, replacing the proof lost with the thin core on 2026-08-27. Note it proves the SIDEBAND gate, not a parity gate — a mixed pairing carries no parity at all here, so there is no parity property left to state. The generated variants are instantiated with `ENABLE_PARITY=0`, and the APB4-port never-sees-parity question the thin core answered does not arise when the port has no parity pins. Parity itself is covered in simulation by `test_apbx_xbar_parity.py`, which generates an all-APB5 2x2 variant with `enable_parity=True`.

APB5 PWAKEUP Across the Fabric

PWAKEUP is a Requester output (IHI 0024E section 3.7.2, Table 3-7). It is an input on an APB5 master port. On an APB5 slave port, where the fabric is the Requester, it is an output driven by:

```
s<j>_apb_PWAKEUP = <apb5_master boundary IP's registered wake>
                 | m<i>_apb_PWAKEUP  (every APB5 master)
```

The boundary term is high while a command for that slave is pending or a transfer is in flight. It covers APB4 masters, which have no PWAKEUP, and it satisfies the 3.7.2 rule that PWAKEUP stays high until PREADY once it is high with PSEL. The master terms forward a requester's wake ahead of its PSEL. That forwarding is not routed by decode, because every master can reach every slave and the address is not known before PSEL. The OR is unregistered, and 3.7.2 names a glitch-free OR tree as a valid construction. The slave's side carries no wake back to any master. `formal/apbx_xbar/apbx_xbar_2to2_mixed/` proves the forwarding (`ap_pwakeup_forwarded`), the wake under every PSEL (`ap_pwakeup_covers_psel`) and the quiet reset state (`ap_pwakeup_not_fabricated`).

Pre-Generated Variants

Module	M×N	Use Case
`apbx_xbar_1to1`	1×1	Passthrough, protocol conversion
`apbx_xbar_2to1`	2×1	Multi-master arbitration
`apbx_xbar_1to4`	1×4	Simple SoC peripheral bus
`apbx_xbar_2to4`	2×4	Typical SoC with CPU+DMA
`apbx_xbar_2to2_mixed`	2×2	Mixed APB4/APB5 (m0=APB4, m1=APB5; s0=APB5, s1=APB4), per-port version gating

LOC per variant is in HAS ch05_performance/03_resources.md, which is the single source of truth for it. The column that used to sit here went stale within two days of being reconciled.

: Pre-Generated Crossbar Variants

Custom Variants: Generated on-demand via Python script

Design Philosophy

Proven Components: - Built from production-tested `apb4_slave.sv` and `apb4_master.sv` - No new protocol logic - pure composition - Each component independently verified

Scalability: - Parametric generation for any MxN configuration - Resource usage scales linearly with M×N - No centralized bottlenecks

Predictability: - Round-robin arbitration provides deterministic behavior - Fixed address map simplifies software integration - Deterministic transfer cost: 9 pclk cycles SETUP-to-PREADY (10 back-to- back) with one master, 10 (11 back-to-back) when arbitrated

---

Next Steps

- See Address Decode and Arbitration for detailed operation examples
- See PRD.md for complete specification
- See CLAUDE.md for integration guidance
- See README.md for quick start guide

Version: 1.2 **Last Updated:** 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Address Decode and Arbitration

Component: Lattice (MxN Interconnect) **Version:** 1.2 **Status:** Production Ready

Overview

Two mechanisms do the crossbar's work: 1. **Address Decode** - How incoming addresses map to specific slaves 2. **Arbitration** - How multiple masters share access to the same slave

---

Address Decode

Address Map Structure

The shipped variants use a 64KB (0x10000 bytes) region per slave, fixed for a given generated module and chosen at generation via `--slave-size`:

```
BASE_ADDR + 0x0000_0000 → 0x0000_FFFF : Slave 0 (64KB)
BASE_ADDR + 0x0001_0000 → 0x0001_FFFF : Slave 1 (64KB)
BASE_ADDR + 0x0002_0000 → 0x0002_FFFF : Slave 2 (64KB)
BASE_ADDR + 0x0003_0000 → 0x0003_FFFF : Slave 3 (64KB)
...
BASE_ADDR + 0x000F_0000 → 0x000F_FFFF : Slave 15 (64KB)
```

Default BASE_ADDR: 0x10000000

Total Address Space: (number of slaves) × 64KB

Decode Algorithm

The crossbar extracts the slave index from the upper bits of the address offset:

```
offset = PADDR - BASE_ADDR
slave_index = offset[16 +: $clog2(S)]   // ceil(log2(S)) bits above
the 64KB window
```

Out-of-Range Addresses

This applies to the variants that DECODE — `1to4`, `2to4`, `2to2_mixed`. In `1to1` and `2to1` there is a single slave, no address decode, and `BASE_ADDR` is unused: every access is forwarded to that slave whatever its address, and the slave's own response is returned.

For the decoding variants, an access outside `[BASE_ADDR, BASE_ADDR + S x 64KB)` is a **decode miss**. The crossbar accepts it and answers locally with **PSLVERR**; no slave sees the transfer, and the arbiters are not involved. The master's transaction completes normally (PREADY asserts) with the error flagged, so a bad pointer surfaces as an APB error response rather than a hang.

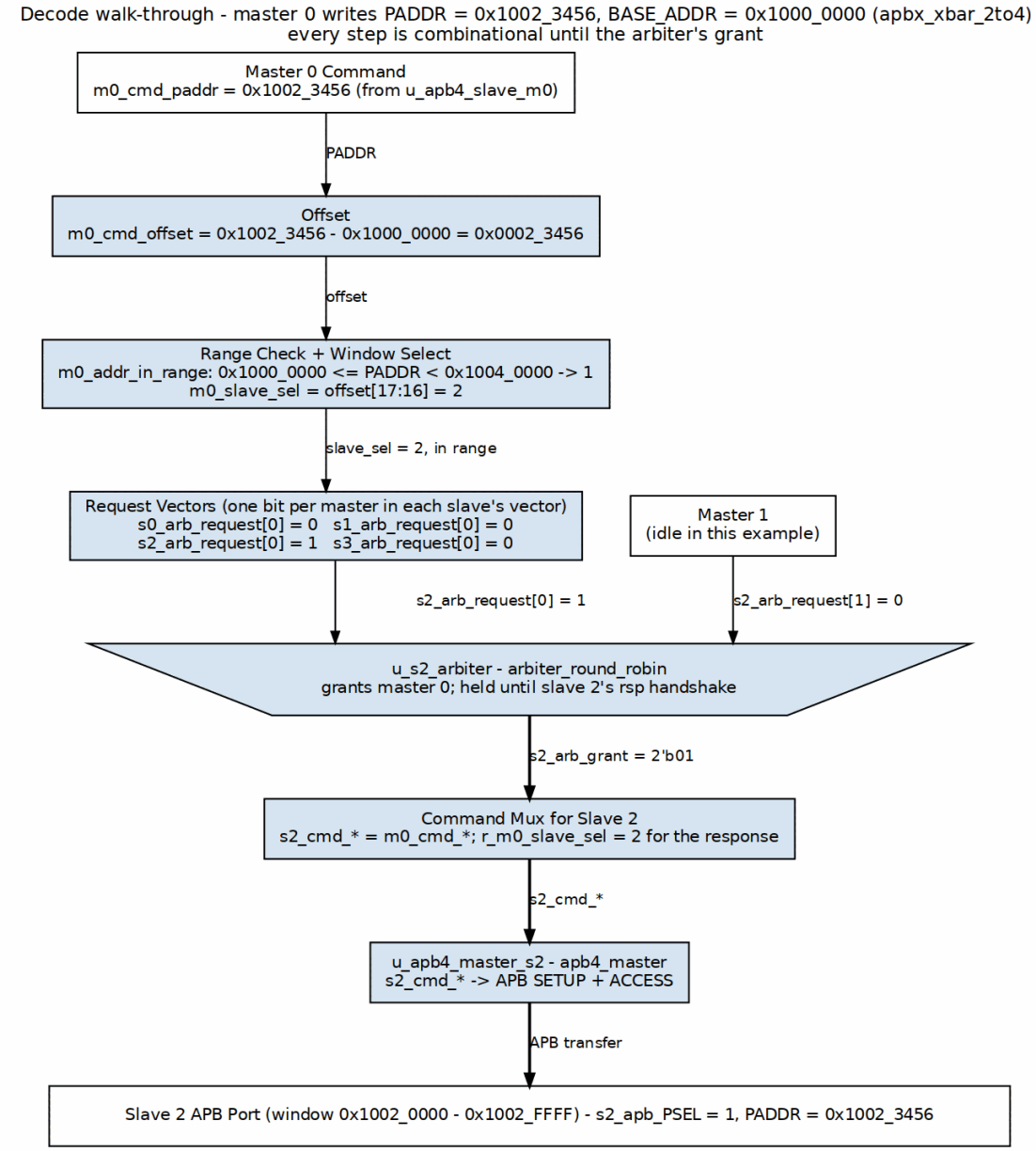
Earlier RTL left `cmd_ready` low on a miss, which wedged the master in ACCESS with PREADY low and no error signature (projects/components/fabric-gen-ip/lattice-apbx-xbar TASK-002, was APBX-002). Regression: the decode-miss scenarios in `test_apbx_xbar_1to4.py`, `test_apbx_xbar_2to4.py`, and `test_apbx_xbar_2to2_mixed.py`.

Why bit 16 upward? - 64KB = 0x10000 = 2^16 bytes - Lower 16 bits (offset[15:0]) are byte address within slave - `ceil(log2(S))` bits starting at bit 16 select the slave: [17:16] for a 4-slave variant, [16] for 2 slaves. The shipped variants wire exactly those bits — `m0_slave_sel` is 2 bits wide on 1to4/2to4, 1 bit on 2to2_mixed, NOT a 4-bit index - The generator supports up to 16 slaves; bits above the index field do not select a slave, they fail the range check and become a decode miss

Address Decode Flow Diagram

The diagram walks one concrete address — 0x10023456 — through the decode to Slave 2:

Figure 2.1: Address Decode Flow



Address Decode Flow

Decode Example Walkthrough

Scenario: Master accesses address 0x10023456 with BASE_ADDR = 0x10000000

Step 1: Calculate Offset

```
offset = PADDR - BASE_ADDR
       = 0x10023456 - 0x10000000
       = 0x00023456
```

Step 2: Extract Slave Index

```
slave_index = offset[17:16]        // S=4 -> 2 bits
            = 0x00023456 >> 16
            = 0x2
```

Step 3: Parallel Decode

The crossbar checks all slave ranges in parallel: - Slave 0: 0x00000 - 0x0FFFF → NO MATCH - Slave 1: 0x10000 - 0x1FFFF → NO MATCH - **Slave 2: 0x20000 - 0x2FFFF → MATCH** - Slave 3: 0x30000 - 0x3FFFF → NO MATCH

Step 4: Route to Arbiter

Transaction request forwarded to Arbiter[2] (Slave 2's arbiter)

Step 5: Forward to Slave

After arbitration grant, transaction forwarded via apb4_master[2] to physical Slave 2

Final Address Sent to Slave: 0x10023456 (full address preserved)

Multiple Address Maps

Different BASE_ADDR values create distinct address maps:

Example: Two Crossbars

```
// Peripheral Bus: 0x1000_0000 - 0x1003_FFFF
apbx_xbar_1to4 #(
    .BASE_ADDR(32'h1000_0000)
) u_periph_xbar (...);
// Slaves at: 0x1000_0000, 0x1001_0000, 0x1002_0000, 0x1003_0000

// Memory-Mapped I/O: 0x8000_0000 - 0x8003_FFFF
apbx_xbar_1to4 #(
    .BASE_ADDR(32'h8000_0000)
) u_mmio_xbar (...);
// Slaves at: 0x8000_0000, 0x8001_0000, 0x8002_0000, 0x8003_0000
```

Arbitration

Per-Slave Round-Robin

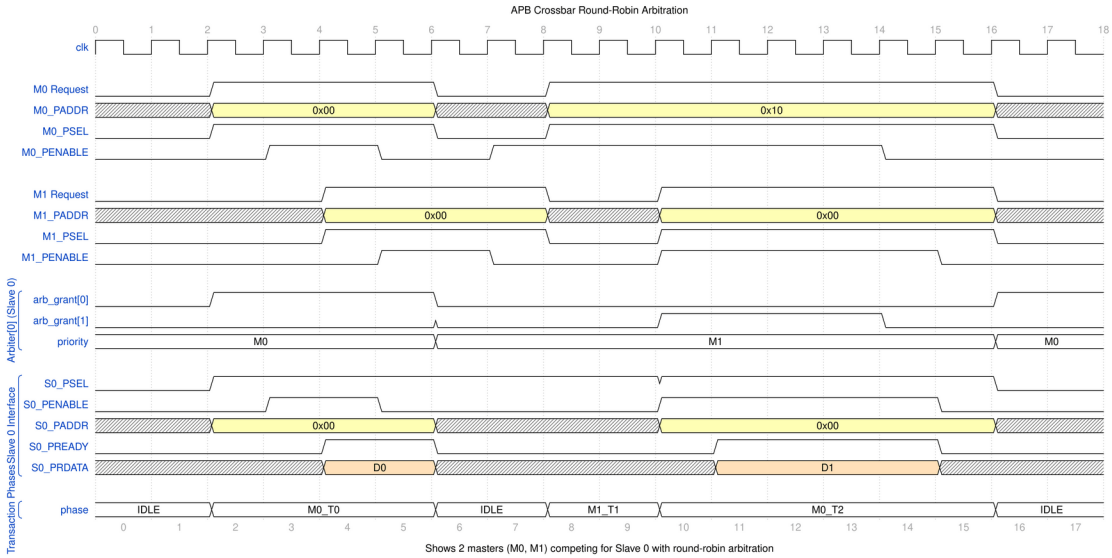
Each slave has an **independent arbiter** that implements round-robin scheduling:

Key Properties: - **Fair:** No master can starve another - **Independent:** Each slave arbitrates separately - **Predictable:** Priority rotates after each grant - **Persistent:** Grant held from command through response

Round-Robin Timing Diagram

Two masters (M0, M1) competing for access to Slave 0:

Waveform 2.1: Round-Robin Arbitration Timing



Round-Robin Arbitration Timing

Arbitration Example Walkthrough

Scenario: Master 0 and Master 1 both want to access Slave 0

Initial State: - Priority: M0 (M0 has priority initially) - Slave 0: IDLE

This walkthrough shows the ORDER of arbitration events, not their timing. It is deliberately not cycle-numbered: a numbered version here showed a three-cycle transaction, which contradicted this chapter's own measured 9-cycle SETUP-to-PREADY figure. `S0_PSEL` cannot assert in the grant cycle — after the REGISTERED grant the command still crosses the `apb4_master` cmd skid, the FSM's IDLE launch and its SETUP state before the downstream PSEL appears. For real numbers see "Transaction Latency" below and `dv/tests/test_apbx_xbar_timing.py`.

Note also that M0_PENABLE is driven by the external master during its own ACCESS phase; the arbiter does not assert it, and it is high before any grant exists.

Transaction 1: M0 requests Slave 0

```
  M0 asserts PSEL with PADDR=0x1000_0000
  Arbiter[0] grants M0 (only requester)
  ... command traverses the fabric ...
  Slave 0 responds; transaction completes
  Priority rotates: M1 now has priority
```

Transaction 2: M0 and M1 both request Slave 0

```
  M0 asserts PSEL with PADDR=0x1000_0010
  M1 asserts PSEL with PADDR=0x1000_0000   -- CONFLICT
  Arbiter[0] grants M1 (has priority); M0 waits
  ... command traverses the fabric ...
  Slave 0 responds; M1's transaction completes
  Priority rotates: M0 now has priority
```

Transaction 3: M0 again, after rotation

```
  M0 is still asserting (it was blocked)
  Arbiter[0] grants M0 (now has priority)
  ... command traverses the fabric ...
  Slave 0 responds; transaction completes
  Priority rotates: M1 now has priority
```

Result: Fair access — each master is served in turn when both request. The grant is held from command acceptance to the response handshake (WAIT_GNT_ACK(1)), so a blocked master waits one full transaction, not one cycle.

Multi-Slave Parallelism

Key Feature: Different slaves can be accessed simultaneously by different masters

Example: Parallel Transactions

```
Time T0:
- Master 0 accesses Slave 0 (UART) - GRANTED
- Master 1 accesses Slave 2 (Timer) - GRANTED
Both transactions proceed in parallel (no conflict)

Time T1:
- Master 0 accesses Slave 0 (UART) - GRANTED
- Master 1 accesses Slave 0 (UART) - BLOCKED (arbitration)
Master 1 waits for Master 0 to complete

Time T2:
- Master 0 completes, releases Slave 0
```

```
- Master 1 accesses Slave 0 (UART) - GRANTED
Priority rotated for Slave 0's arbiter
```

Benefit: Maximum throughput when masters access different slaves

Grant Persistence

Critical Property: Once granted, a master holds the slave until response completes

Why This Matters:

```
WITHOUT Grant Persistence:
- Master 0 granted Slave 0
- Master 0 asserts PENABLE
- *Grant could change here* ← BREAKS PROTOCOL!
- Slave 0 responds to wrong master

WITH Grant Persistence:
- Master 0 granted Slave 0
- Master 0 asserts PENABLE
- Grant held until PREADY asserted ← SAFE
- Slave 0 responds to correct master
- Grant released for next transaction
```

Implementation: Grant signal registered and held from PSEL assertion through PREADY response

Arbitration Latency

Best Case (No Contention): - 1 cycle: the grant is REGISTERED (`grant <= w_next_grant;` in `arbiter_round_robin`), so request → grant is never same-cycle - Fabric overhead is the dominant term, not APB's 2-cycle minimum: a full transfer measures **10** cycles SETUP-to-PREADY on an arbitrated variant (see "Transaction Latency" below). Everything in this chapter is an M > 1 figure — there is no arbiter at all when M = 1, and those variants are a cycle faster.

Worst Case (Maximum Contention): - Wait for the masters ahead in the round-robin to complete FULL transactions — the grant is held until the response handshake (`WAIT_GNT_ACK(1)`), so each is a full transaction, not 1 cycle. The right figure for a QUEUED transaction is the back-to-back period of **11** cycles, not the 10-cycle single-transfer latency — a waiting master inherits the same mandatory SETUP cycle that separates any two consecutive transfers - Example: 2 masters, worst case ~= one full transaction (11 cycles) of wait - Example: 4 masters, worst case = 3 full transactions ~= 33 cycles

Average Case (Random Access Pattern): - (M-1)/2 full transactions of average wait (~11 cycles each) - Statistical fairness over time

---

Integration Examples

Example 1: CPU to 4 Peripherals (No Contention)

```
apbx_xbar_1to4 #(
    .BASE_ADDR(32'h1000_0000)
) u_periph_xbar (
    // Single master (CPU)
    // .m0_apb_PSEL(cpu_apb_PSEL), .m0_apb_PENABLE(...), ...  (elided)

    // 4 slaves (UART, GPIO, Timer, SPI)
    .s0_apb_* (uart_*),   // 0x1000_0000
    .s1_apb_* (gpio_*),   // 0x1001_0000
    .s2_apb_* (timer_*),  // 0x1002_0000
    .s3_apb_* (spi_*)     // 0x1003_0000
);
```

Behavior: - No arbitration needed (single master) - Pure address decode functionality - Zero arbitration overhead

Example 2: CPU + DMA to 4 Peripherals (Potential Contention)

```
apbx_xbar_2to4 #(
    .BASE_ADDR(32'h4000_0000)
) u_soc_xbar (
    // Two masters
    // .m0_apb_PSEL(cpu_apb_PSEL), .m0_apb_PENABLE(...), ...  (elided)
    // .m1_apb_PSEL(dma_apb_PSEL), ...                        (elided)

    // 4 slaves
    .s0_apb_* (mem_ctrl_*),  // 0x4000_0000
    .s1_apb_* (uart_*),      // 0x4001_0000
    .s2_apb_* (i2c_*),       // 0x4002_0000
    .s3_apb_* (adc_*)        // 0x4003_0000
);
```

Behavior: - Each slave has independent arbiter - CPU and DMA can access different slaves simultaneously - If both access same slave, round-robin arbitration - Fair access guaranteed

---

Performance Characteristics

Throughput

Single Master: - Back-to-back transactions supported, but NOT overlapped: `apb4_slave` is a one-command-at-a-time FSM, so the next command is captured only after the previous transaction completes - Measured sustained cadence: 1 transfer per **11** pclk cycles (PREADY-to-PREADY) at an always-ready slave, on an arbitrated variant. That is one MORE than its 10-cycle SETUP-to-PREADY single-transfer latency: nothing overlaps, and the next transfer's mandatory SETUP cycle cannot share a cycle with the previous transfer's ACCESS. (An earlier "zero bubble" claim described a pipeline this RTL does not have; a later "cadence equals latency" claim dropped the SETUP cycle.)

Multiple Masters (Same Slave): - Round-robin introduces fair sharing - Each master gets ~1/M of bandwidth - Example: 2 masters = 50% bandwidth each

Multiple Masters (Different Slaves): - Full parallelism - Each master gets 100% of its target slave - Total system bandwidth = (number of slaves) × slave_bandwidth

Latency

Components:

1. **Address Decode:** 0 cycles (combinational, parallel)
2. **Arbitration Decision:** 1 cycle (grant is registered)
3. **Master APB phases:** 2 cycles (PSEL, then PENABLE)
4. **Boundary IP + skid buffers:** ~6 cycles — `apb4_slave` capture, registered cmd skid, `apb4_master` IDLE→SETUP→ACCESS, registered rsp skid, `apb4_slave` BUSY→PREADY
5. **Slave Response:** Variable (adds to the above)

Total Minimum Latency: 10 cycles SETUP-to-PREADY (9 of them ACCESS->PREADY) on an arbitrated variant, uncontended with a zero-wait slave — measured on `apbx_xbar_2to1`, and the same on `2to4` and `2to2_mixed`. A single-master variant has no arbiter and measures 9 (8 fabric). APB's 2-cycle minimum applies to a directly-attached slave; it does not apply through this fabric, which converts APB→cmd/rsp→APB across registered buffers in both directions.

Sustained cadence is 11 cycles, PREADY-to-PREADY, not 10 — see "Single Master" above and HAS 5.2 for why the two differ and for the M = 1 column.

With Contention: add one full transaction (~11 cycles) per master ahead in the round-robin

---

Design Notes

Why 64KB Per Slave?

Rationale: - Sufficient for most APB peripherals (typically 256B - 4KB register space) - Simple decode logic (single shift operation) - Allows up to 16 slaves with clean byte alignment - Software-friendly (each peripheral has “round” base address)

Alternatives: - Smaller regions (4KB, 16KB) → more slaves, more complex decode - Larger regions (256KB, 1MB) → fewer slaves, wasted address space

Why Round-Robin?

Advantages: - Fair: Prevents starvation - Simple: Minimal logic (counter + comparator) - Predictable: Software can reason about worst-case latency - No configuration: Works out-of-box

Alternatives: - Fixed priority → Can starve low-priority masters - Weighted fair queuing → More complex, requires configuration - Lottery scheduling → Unpredictable, harder to verify

---

Troubleshooting

Issue: Wrong Slave Selected

Check: 1. BASE_ADDR parameter correct? 2. Address within expected range? 3. Calculated slave_index matches expectation?

Debug:

```
offset = PADDR - BASE_ADDR
slave_index = offset >> 16
expected_slave = slave_index  // Should match actual PSEL
```

Issue: Master Starved

Check: 1. Other masters continuously accessing same slave? 2. Round-robin priority rotating correctly? (grant is released at the RESPONSE handshake — `grant_ack = grant && rsp_valid && rsp_ready` with `WAIT_GNT_ACK(1)` — so a master that never completes a transaction holds its grant) 3. Is the starved master's address actually decoding in range? An out-of-range access completes locally with PSLVERR and never reaches an arbiter.

There is no timeout or monitoring knob to enable: `arbiter_round_robin` has no timeout logic, parameter, or port. An earlier revision of this checklist referenced one.

Expected Behavior: - Each master should get grant within M transactions - No master should wait indefinitely

Issue: Back-to-Back Transactions Stalling

Check: 1. Grant persistence working? 2. Slave asserting PREADY correctly? 3. No unintended pipeline bubbles?

Expected Behavior: - Consecutive transactions from same master should flow without gaps - Only arbitration conflicts should introduce wait states

---

Next Steps

- See Architecture for top-level design overview
- See PRD.md for complete specification
- See CLAUDE.md for integration guidance
- See README.md for quick start guide

Version: 1.2 **Last Updated:** 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Lattice RTL Generator

Component: Lattice (MxN Interconnect) **Version:** 1.2 **Status:** Production Ready

Overview

Five pre-generated variants (1to1, 2to1, 1to4, 2to4, 2to2_mixed) cover the common configurations. For anything else, the Python-based generator produces any arbitrary MxN crossbar up to 16x16.

Philosophy: - Pre-generated variants for common use cases (fast integration) - Python generator for custom configurations (flexibility) - Template-based code generation (consistency) - Built from proven components (quality)

Generator Architecture

Two-Level Structure

```
projects/components/fabric-gen-ip/lattice-apbx-xbar/
├── bin/
│   └── generate_xbars.py              ← Convenience wrapper script
│
└── projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
    └── apbx_xbar_generator.py          ← Core generator engine
```

Why Two Files? 1. **Core Generator** (`apbx_xbar_generator.py`): Reusable library function - Can be imported by other scripts - Contains `generate_apbx_xbar()` function - Located in the component's own `bin/` directory: `projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/apbx_xbar_generator.py`. It is NOT under `bin/rtl_generators/` — that path was claimed here until 2026-08-29 and contradicted this chapter's own tree, its troubleshooting section, the README and the PRD. `bin/rtl_generators/amba/` contains only an `__init__.py`.

2. **Convenience Wrapper** (`generate_xbars.py`): Project-specific script
   - Easy command-line interface
   - Pre-configured for standard variants
   - Located in project `bin/` directory

Quick Start

Generate Standard Variants (1:1, 2:1, 1:4, 2:4)

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py
```

Output:

```
Generating 1-to-1 crossbar...
  apbx_xbar_1to1.sv
Generating 2-to-1 crossbar...
  apbx_xbar_2to1.sv
Generating 1-to-4 crossbar...
  apbx_xbar_1to4.sv
Generating 2-to-4 crossbar...
  apbx_xbar_2to4.sv
Generating 2-to-2_mixed crossbar...
  apbx_xbar_2to2_mixed.sv

Generated 5 crossbar variants
```

Generate Custom Variant

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6
```

Output:

```
Generating 3-to-6 crossbar...
Generated apbx_xbar_3to6.sv
```

Generate with Custom Base Address

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 4 --slaves 8 --base-addr 0x80000000
```

Output:

```
Generating 4-to-8 crossbar...
Generated apbx_xbar_4to8.sv

Address Map:
  Slave 0: [0x80000000, 0x8000FFFF]
  Slave 1: [0x80010000, 0x8001FFFF]
  Slave 2: [0x80020000, 0x8002FFFF]
  ...
  Slave 7: [0x80070000, 0x8007FFFF]
```

Command-Line Interface

Full Options

```
python generate_xbars.py [OPTIONS]
```

Options:

Command Line Options

Option	Short	Type	Default	Description
`--masters`	`-m`	int	None	Number of master interfaces (1-16)
`--slaves`	`-s`	int	None	Number of slave interfaces (1-16)
`--base-addr`	`-b`	hex	0x10000000	Base address for slave address map
`--slave-size`	-	hex	0x10000	Address space per slave: 0x1000=4KB, 0x10000=64KB (matches shipped variants)
`--help`	`-h`	-	-	Show help message

The CLI covers the common cases. Per-port APB version and parity are reachable only through the Python API (`generate_apbx_xbar`), since they take per-port lists rather than scalars:

Python API Arguments

Argument	Type	Default	Description
`slave_size`	int	0x1000	Address window per slave. `generate_xbars.py` passes 0x10000 for the shipped variants
`master_versions`	list	None	Per-master APB version (4/5); None ⇒ all APB4
`slave_versions`	list	None	Per-slave APB version (4/5); None ⇒ all APB4
`name_suffix`	str	`""`	Appended to the generated module name

Argument	Type	Default	Description
enable_parity	bool	False	Carry APB5 parity (IHI 0024E ch. 5 *CHK signals, odd) on APB5 ports

enable_parity checks and regenerates; it does not pass through. A generated variant **checks** parity at the ingress boundary and **regenerates** it on the far side, because the parity bits do not cross the cmd/rsp interface the boundary IP deconstructs each transfer into. The cmd/rsp fabric between the two is outside the protected domain, so with enable_parity=True each **APB5** port exposes its own parity_error_* outputs (m0_parity_error_wdata, s0_parity_error_rdata, …) — a check whose result goes nowhere is not protection. APB4 ports get none, and the shipped apbx_xbar_2to2_mixed is generated with parity off, so there the boundary IP's flags are tied off internally. A mixed APB4/APB5 pairing carries no parity at all.

(The retired apbx_xbar_thin passed parity through end to end instead, which is why this distinction used to be drawn against it. It was deleted on 2026-08-27; there is no other family to contrast with now.)

Rules: - If neither --masters nor --slaves specified → Generate all standard variants - If only one specified → Error (must specify both) - If both specified → Generate custom variant

Examples

Example 1: Generate All Standard Variants

```
python generate_xbars.py
```

Example 2: Generate 3x6 Crossbar

```
python generate_xbars.py --masters 3 --slaves 6
```

Example 3: Generate 5x10 Crossbar with Custom Base

```
python generate_xbars.py -m 5 -s 10 -b 0xA0000000
```

Example 4: Show Help

```
python generate_xbars.py --help
```

Generated Code Structure

Template Components

The generator creates a complete SystemVerilog module with:

1. **Header with Address Map**

```
// 2-to-4 APB crossbar with address decoding and arbitration
// 2 masters to 4 slaves using apb4_slave and apb4_master modules
//
// (This walkthrough shows the SHIPPED apbx_xbar_2to4 so every
constant
//  below is real (2to4). For a 3-to-6 the shapes scale: 3-bit
slave_sel,
//  a 0x00060000 range bound, and CLIENTS(3) arbiters.)
//
// Address Map (same for all masters):
//   Slave 0: [0x10000000, 0x1000FFFF]
//   Slave 1: [0x10010000, 0x1001FFFF]
//   Slave 2: [0x10020000, 0x1002FFFF]
//   Slave 3: [0x10030000, 0x1003FFFF]
```

2. **Module Declaration with Parameters**

```
module apbx_xbar_2to4 #(
    parameter int ADDR_WIDTH = 32,
    parameter int DATA_WIDTH = 32,
    parameter int STRB_WIDTH = DATA_WIDTH / 8,
    parameter logic [ADDR_WIDTH-1:0] BASE_ADDR = 32'h10000000
) (
    // Clock and Reset
    input  logic                  pclk,
    input  logic                  presetn,

    // Master 0-1 APB interfaces
    // Slave 0-3 APB interfaces
);
```

3. **Internal Signal Declarations**

```
    // Command/Response interfaces for master 0-1 apb4_slave
    logic m0_cmd_valid, m0_cmd_ready, m0_cmd_pwrite;
    logic [ADDR_WIDTH-1:0] m0_cmd_paddr;
    // ... (full cmd/rsp bus for each master)

    // Command/Response interfaces for slave 0-3 apb4_master
    logic s0_cmd_valid, s0_cmd_ready, s0_cmd_pwrite;
    logic [ADDR_WIDTH-1:0] s0_cmd_paddr;
    // ... (full cmd/rsp bus for each slave)
```

4. **Master-Side apb4_slave Instantiations**

```
    // Master 0 apb4_slave
    apb4_slave #(
        .ADDR_WIDTH(ADDR_WIDTH),
        .DATA_WIDTH(DATA_WIDTH)
    ) u_apb4_slave_m0 (
        .pclk(pclk),
        .presetn(presetn),
        // APB interface
        .s_apb_PSEL(m0_apb_PSEL),
        .s_apb_PENABLE(m0_apb_PENABLE),
        // ... (full APB connections)
        // cmd/rsp interface
        .cmd_valid(m0_cmd_valid),
        .cmd_ready(m0_cmd_ready),
        // ... (full cmd/rsp connections)
    );

    // Master 1 apb4_slave instance...
```

5. **Address Decode Logic**

```
    // Per-master offset decode (verbatim shape of the real output --
    // there is no request-matrix signal; per-slave cmd_valid terms
are
    // built directly from the decoded index)
    logic [ADDR_WIDTH-1:0] m0_cmd_offset;
    logic [1:0] m0_slave_sel;
    logic m0_addr_in_range;

    always_comb begin
        m0_cmd_offset    = m0_cmd_paddr - BASE_ADDR;
        m0_addr_in_range = (m0_cmd_paddr >= BASE_ADDR) &&
                           (m0_cmd_paddr < (BASE_ADDR +
32'h00040000));
        m0_slave_sel     = m0_cmd_offset[17:16];   // ceil(log2(S))
bits
    end

    assign s0_cmd_valid = m0_cmd_valid && m0_addr_in_range &&
(m0_slave_sel == 2'd0);
    assign s1_cmd_valid = m0_cmd_valid && m0_addr_in_range &&
(m0_slave_sel == 2'd1);
    // ... one per slave. For M>1 these terms are NOT repeated per
    // master onto the same signal -- the per-slave arbiter grant
    // selects which master drives it:
    //   case (1'b1)
    //     s0_arb_grant[0]: s0_cmd_valid = m0_cmd_valid &&
```

```
m0_addr_in_range && (m0_slave_sel == 2'd0);
    //     s0_arb_grant[1]: s0_cmd_valid = m1_cmd_valid &&
m1_addr_in_range && (m1_slave_sel == 2'd0);
    //   endcase
```

6. **Per-Slave Arbitration Logic**

```
    // One arbiter instance per slave (no hand-rolled priority
    // counters in the output -- the generator instantiates the
    // library arbiter):
    arbiter_round_robin #(
        .CLIENTS(2),          // = M (3 for a 3-to-6)
        .WAIT_GNT_ACK(1)      // hold grant until the response
handshake
    ) u_s0_arbiter (
        .clk(pclk), .rst_n(presetn), .block_arb(1'b0),
        .request(s0_arb_request), .grant_ack(s0_arb_grant_ack),
        .grant(s0_arb_grant),
        .grant_valid(), .grant_id(), .last_grant()
    );
```

7. **Response Routing Logic**

```
    // Route responses back to requesting masters
    always_comb begin
        // Track which master initiated each transaction
        // Route PRDATA/PSLVERR back to correct master
    end
```

8. **Slave-Side apb4_master Instantiations**

```
    // Slave 0 apb4_master
    apb4_master #(
        .ADDR_WIDTH(ADDR_WIDTH),
        .DATA_WIDTH(DATA_WIDTH)
    ) u_s0_master (
        .pclk(pclk),
        .presetn(presetn),
        // cmd/rsp interface
        .cmd_valid(s0_cmd_valid),
        .cmd_ready(s0_cmd_ready),
        // ... (full cmd/rsp connections)
        // APB interface -- apb4_master's ports are m_apb_*, so the
        // slave-side port sN_apb_* connects to m_apb_* on the
instance
        .m_apb_PSEL     (s0_apb_PSEL),
        .m_apb_PENABLE  (s0_apb_PENABLE),
        // ... (full APB connections)
    );

    // Slave 1-3 apb4_master instances...
```

9. Module End

```
endmodule : apbx_xbar_2to4
```

Generator Algorithm

Step-by-Step Process

1. Validate Parameters

```
if M < 1 or M > 16:
    raise ValueError(f"Number of masters must be 1-16, got {M}")
if N < 1 or N > 16:
    raise ValueError(f"Number of slaves must be 1-16, got {N}")
```

2. Calculate Derived Parameters

```
slave_addr_bits = max(1, math.ceil(math.log2(N)))  # Bits for slave
index
strb_width = data_width // 8                        # Byte strobe
width
module_name = f"apbx_xbar_{M}to{N}"                 # Module name
```

3. Generate Address Map Documentation

```
for s in range(N):
    addr_offset = s * 0x10000  # 64KB per slave
    addr_start = base_addr + addr_offset
    addr_end = addr_start + 0xFFFF
    # Generate comment: Slave S: [START, END]
```

4. Generate Module Declaration - Parameters: ADDR_WIDTH, DATA_WIDTH, STRB_WIDTH, BASE_ADDR - Ports: Clock, reset - Master interfaces: M × full APB interface - Slave interfaces: N × full APB interface

5. Generate Internal Signals - Master cmd/rsp buses: M sets - Slave cmd/rsp buses: N sets - Request matrix: [M][N] - Grant matrix: [N][M]

(No priority counters are emitted. Rotation state lives inside the `arbiter_round_robin` instances the generator wires up, which is what the code sample further down says.)

6. Instantiate Master-Side Components

```
for m in range(M):
    # Generate apb4_slave instance
    # Connect APB interface to external master m
    # Connect cmd/rsp to internal buses
```

7. Generate Address Decode Logic

```
for m in range(M):
    # For each master, decode address to slave index
    # offset = paddr - BASE_ADDR
    # slave_idx = offset[16 +: ceil(log2(S))]  # S-sized index above
```

```
64KB
    # Set m_to_s_req[m][slave_idx] = 1
```

8. Generate Arbitration Logic

```
for s in range(N):
    # For each slave, arbitrate among M masters
    # Round-robin: priority rotates after each grant
    # Grant persistence: hold until transaction completes
```

9. Generate Response Routing

```
# Track transaction source (which master)
# Route responses back to originating master
```

10. Instantiate Slave-Side Components

```
for s in range(N):
    # Generate apb4_master instance
    # Connect cmd/rsp from internal buses
    # Connect APB interface to external slave s
```

11. Write Generated Code

```
with open(output_file, 'w') as f:
    f.write(generated_code)
```

Code Generation Patterns

Loop-Based Generation

Master Interfaces:

```
for m in range(M):
    code += f"    // Master {m} APB interface\n"
    code += f"    input  logic m{m}_apb_PSEL,\n"
    code += f"    input  logic m{m}_apb_PENABLE,\n"
    # ... (all APB signals for master m)
```

Slave Interfaces:

```
for s in range(N):
    code += f"    // Slave {s} APB interface\n"
    code += f"    output logic s{s}_apb_PSEL,\n"
    code += f"    output logic s{s}_apb_PENABLE,\n"
    # ... (all APB signals for slave s)
```

String Formatting

Hex Values:

```
code += f"    parameter logic [ADDR_WIDTH-1:0] BASE_ADDR =
{addr_width}'h{base_addr:08X}\n"
# Example: 32'h10000000
```

Address Map Comments:

```
addr_start = base_addr + (s * 0x10000)
code += f"//   Slave {s}: [0x{addr_start:08X}, 0x{addr_start +
0xFFFF:08X}]\n"
# Example: Slave 2: [0x10020000, 0x1002FFFF]
```

Customization Examples

Example 1: Large SoC with 10 Peripherals

Scenario: CPU + DMA need to access 10 peripherals

Command:

```
python generate_xbars.py --masters 2 --slaves 10
```

Generated File: `apbx_xbar_2to10.sv`

Address Map:

```
Slave 0:  0x10000000 - 0x1000FFFF (UART0)
Slave 1:  0x10010000 - 0x1001FFFF (UART1)
Slave 2:  0x10020000 - 0x1002FFFF (GPIO)
Slave 3:  0x10030000 - 0x1003FFFF (I2C)
Slave 4:  0x10040000 - 0x1004FFFF (SPI)
Slave 5:  0x10050000 - 0x1005FFFF (Timer0)
Slave 6:  0x10060000 - 0x1006FFFF (Timer1)
Slave 7:  0x10070000 - 0x1007FFFF (PWM)
Slave 8:  0x10080000 - 0x1008FFFF (ADC)
Slave 9:  0x10090000 - 0x1009FFFF (Watchdog)
```

LOC: ~2500 lines (extrapolated, not generated and counted — the measured anchors are in HAS ch05_performance/03_resources.md)

Example 2: Multi-CPU System

Scenario: 4 CPUs accessing shared peripheral bus with 4 slaves

Command:

```
python generate_xbars.py --masters 4 --slaves 4
```

Generated File: `apbx_xbar_4to4.sv`

Benefits: - Fair arbitration among 4 CPUs - Each slave independently arbitrated - Maximum throughput when CPUs access different slaves

LOC: ~1600 lines (extrapolated, see above)

Example 3: Memory-Mapped I/O Region

Scenario: Single master accessing I/O region at 0xA0000000

Command:

```
python generate_xbars.py --masters 1 --slaves 8 --base-addr 0xA0000000
```

Generated File: `apbx_xbar_1to8.sv`

Address Map:

```
Slave 0:  0xA0000000 - 0xA000FFFF
Slave 1:  0xA0010000 - 0xA001FFFF
...
Slave 7:  0xA0070000 - 0xA007FFFF
```

Using Generated Modules

Integration Example

```
// Generated file: apbx_xbar_3to6.sv

module my_soc (
    input  logic clk,
    input  logic rst_n,
    // ... other interfaces
);

    // Instantiate 3x6 crossbar
    apbx_xbar_3to6 #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32),
        .BASE_ADDR(32'h10000000)  // Can override at instantiation
    ) u_periph_xbar (
        .pclk(clk),
        .presetn(rst_n),

        // Connect 3 masters (CPU, DMA, Debug)
        .m0_apb_PSEL(cpu_apb_psel),
        .m0_apb_PENABLE(cpu_apb_penable),
        // ... (full CPU APB interface)

        .m1_apb_PSEL(dma_apb_psel),
        .m1_apb_PENABLE(dma_apb_penable),
        // ... (full DMA APB interface)

        .m2_apb_PSEL(debug_apb_psel),
        .m2_apb_PENABLE(debug_apb_penable),
        // ... (full Debug APB interface)

        // Connect 6 slaves (UART, GPIO, Timer, SPI, I2C, ADC)
        .s0_apb_PSEL(uart_psel),
        .s0_apb_PENABLE(uart_penable),
        // ... (full UART APB interface)

        .s1_apb_PSEL(gpio_psel),
        // ... (remaining slaves)
    );

endmodule : my_soc
```

Parameter Override

```
// Use different base address at instantiation
apbx_xbar_3to6 #(
    .BASE_ADDR(32'h80000000)  // Override default 0x10000000
) u_mmio_xbar (
    // ... connections
);
// Slaves now at: 0x80000000, 0x80010000, 0x80020000, ...
```

Generator Limitations and Future Work

Current Limitations

1. **Fixed 64KB Per Slave**
   - Cannot customize individual slave region sizes
   - Workaround: Use multiple crossbars with different BASE_ADDR
2. **Max 16x16 Configuration**
   - Practical limit due to arbitration complexity
   - Workaround: Hierarchical crossbars
3. **Single Address Map**
   - All masters see same slave address map
   - Workaround: Address translation before crossbar
4. **No Priority Levels**
   - Round-robin only, no fixed priority or weighted arbitration
   - Workaround: External priority arbitration before crossbar

Future Enhancements

1. Variable Slave Sizes

```
# Potential future API
generate_apbx_xbar(
    num_masters=2,
    slave_sizes=[4096, 65536, 1024, 16384],  # Custom sizes per slave
    base_addr=0x10000000
)
```

2. Arbitration Policies

```
# Potential future API
generate_apbx_xbar(
    num_masters=4,
    num_slaves=4,
    arbitration="weighted",  # "round-robin", "fixed-priority",
"weighted"
    weights=[4, 2, 1, 1]     # Master 0 gets 4x priority
)
```

3. Address Translation

```
# Potential future API
generate_apbx_xbar(
    num_masters=2,
    num_slaves=4,
```

```
    master_views=[
        {"base": 0x00000000, "remap": True},   # Master 0 sees 0x0
        {"base": 0x10000000, "remap": False}   # Master 1 sees
0x1000_0000
    ]
)
```

Testing Generated Code

Verification Flow

1. Generate Crossbar

```
python generate_xbars.py --masters 2 --slaves 4
```

2. Lint Generated RTL

```
cd ../../rtl/
verilator --lint-only apbx_xbar_2to4.sv
```

3. Run Functional Tests

```
cd ../../dv/tests/
make run-apbx_xbar_2to4-gate
```

4. View Waveforms (If Test Fails)

```
make run-apbx_xbar_2to4-gate-waves
# create_view_cmd() writes the gtkwave command beside the log
```

Recommended Test Cases

1. **Single Master Access** - Verify basic routing
2. **Multi-Master Contention** - Verify arbitration
3. **Back-to-Back Transactions** - Verify consecutive transfers complete at the expected 10/11-cycle cadence
4. **Address Decode Boundary** - Verify slave selection at boundaries
5. **Error Response Propagation** - Verify PSLVERR routing
6. **APB5 Sideband Gating** - Verify `apbx_xbar_2to2_mixed` sideband behavior
7. **Timing Compliance** - Verify published latency/cadence with `test_apbx_xbar_timing`

Advanced Usage

Using as Python Library

```
# Import generator function
# generator lives in the component's bin/, so put it on the path first
import sys; sys.path.insert(0,
'projects/components/fabric-gen-ip/lattice-apbx-xbar/bin')
from apbx_xbar_generator import generate_apbx_xbar

# Generate crossbar programmatically
code = generate_apbx_xbar(
    num_masters=3,
    num_slaves=8,
    base_addr=0xA0000000,
    addr_width=32,
    data_width=32,
    output_file="my_custom_xbar.sv"
)

# Generated code is returned as string
print(f"Generated {len(code)} bytes of SystemVerilog code")

# Can also write to file manually
with open("custom_output.sv", "w") as f:
    f.write(code)
```

Batch Generation

```
# Generate multiple variants in one script
variants = [
    (1, 4, 0x10000000),
    (2, 4, 0x20000000),
    (3, 6, 0x30000000),
    (4, 8, 0x40000000),
]

for masters, slaves, base_addr in variants:
    code = generate_apbx_xbar(
        num_masters=masters,
        num_slaves=slaves,
        base_addr=base_addr,
        output_file=f"xbar_{masters}to{slaves}_0x{base_addr:08X}.sv"
    )
    print(f"Generated xbar_{masters}to{slaves}_0x{base_addr:08X}.sv")
```

Troubleshooting

Issue: "ModuleNotFoundError: No module named 'amba.apbx_xbar_generator'"

Cause: Python can't find the core generator module

Solution:

```
# Verify generator exists
ls -la
projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/apbx_xbar_gene
rator.py

# Run from bin directory
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py
```

Issue: Generated Code Won't Lint

Cause: Generator bug or unsupported configuration

Solution: 1. Check parameter ranges (1-16 for M and N) 2. No span alignment is required — the slave index comes from the OFFSET. The one illegal region is the top S x 64KB of the address space, where `BASE_ADDR + S*64KB` wraps 32-bit and every access becomes a decode miss 3. Report bug with command that caused issue

Issue: Generated Code Too Large

Cause: Very large MxN configuration (e.g., 16x16)

Expected Behavior: - 1x1: 167 LOC (measured) - 2x4: 792 LOC (measured) - 16x16: ~25,000 LOC (extrapolated; the practical limit)

Solution: - Consider hierarchical approach (multiple smaller crossbars) - Or use different interconnect topology (bus, tree, etc.)

References

- Architecture Overview
- Address Decode and Arbitration
- PRD.md - Complete specification
- CLAUDE.md - Integration guidance
- README.md - Quick start guide

Version: 1.2 **Last Updated:** 2026-10-12 **Maintained By:** RTL Design Sherpa Project

Verification Wrappers

`rtl/wrappers/*_wrap.sv` holds four modules — `apbx_xbar_1to1_wrap`, `apbx_xbar_1to4_wrap`, `apbx_xbar_2to1_wrap`, `apbx_xbar_2to4_wrap`.

The generator does not emit them. They are hand-written testbench scaffolding, and a variant gets one only when a test needs it. The clearest evidence is `apbx_xbar_2to2_mixed`: it is generator output like every other variant, and it has no wrapper at all, because its test drives the DUT directly. Regenerating the family produces no wrappers; generating a new custom MxN produces none either.

They are also not design entities and must not be instantiated in a system. Each declares only `pclk` and `presetn` as ports, holds every master- and slave-side APB signal as an internal net, and instantiates the crossbar against those nets. The cocotb testbenches use the wrapper as `toplevel` and drive the bus by hierarchical name.

Keeping the bus off the port list is deliberate: the AXI/APB factory helpers match signals by name pattern, and a wrapper that re-exported `m0_apb_*` and `s0_apb_*` at its own boundary would give those patterns two matches per signal. The internal-net form leaves exactly one.

Verification Wrapper Parameters

Parameter	Default	On which wrappers	Description
`ADDR_WIDTH`	32	all four	Forwarded to the wrapped crossbar
`DATA_WIDTH`	32	all four	Forwarded to the wrapped crossbar
`BASE_ADDR`	`32'h1000_0000`	`1to4`, `2to4` only	Forwarded to the crossbar's decode base. Absent on `1to1`/`2to1`, matching those variants, where `BASE_ADDR` is inert because there is nothing to decode

Lattice Hardware Architecture Specification Index

Version: 1.2 **Date:** 2026-10-12 **Purpose:** High-level hardware architecture specification for Lattice component

---

Document Organization

Note: All chapters are linked below; this index doubles as the manifest for automated document generation.

Front Matter

- Document Information

Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

Chapter 3: Architecture

- Block Diagram
- Data Flow
- Address Mapping

Chapter 4: Interfaces

- Master-Side APB Interfaces
- Slave-Side APB Interfaces
- Clock and Reset

Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resource Estimates

Chapter 6: Integration

- System Requirements
- Parameter Configuration
- Verification Strategy

Related Documentation

- **Lattice MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Product Requirements Document (PRD)

Lattice Generator

Version: 1.0 **Date:** 2025-10-19 **Status:** Production Ready **Owner:** RTL Design Sherpa Project **Parent Document:** `/PRD.md` **Codename:** lattice — directory `lattice-apbx-xbar` (fabric-gen-ip family with `bridge` and `delta`); RTL modules keep the `apbx_xbar_*` prefix

---

1. Executive Summary

The **Lattice** is a parametric generator for configurable MxN APB interconnect fabrics connecting multiple APB masters to multiple APB slaves. It is built from the proven `apb4_slave` and `apb4_master` modules: independent round-robin arbitration per slave, automatic address-based routing, and no new protocol logic.

1.1 Quick Stats

- **Modules:** 5 pre-generated variants + generator for custom sizes
- **Max Capacity:** Up to 16x16 (configurable)
- **Architecture:** apb4_slave → arbitration + decode → apb4_master
- **Status:** Production ready, all tests passing
- **Generator:** Python-based parametric code generation

1.2 Project Goals

- **Primary:** Provide production-quality APB interconnect for SoC integration
- **Secondary:** Demonstrate parametric RTL generation best practices
- **Tertiary:** Support both fixed and dynamically-generated variants

2. Key Design Principles

2.1 Architecture Philosophy

Proven Building Blocks: - Uses `apb4_slave.sv` and `apb4_master.sv` from rtl/amba/apb4 - Each module independently tested and production-proven - Crossbar = composition of proven components

Parametric Generation: - Generator creates any MxN configuration - Pre-generated common variants (1to1, 2to1, 1to4, 2to4, 2to2_mixed) - Custom variants generated on-demand

Clean Separation: - Master-side: APB slaves convert protocol to cmd/rsp - Internal: Arbitration + address decoding - Slave-side: APB masters convert cmd/rsp back to protocol

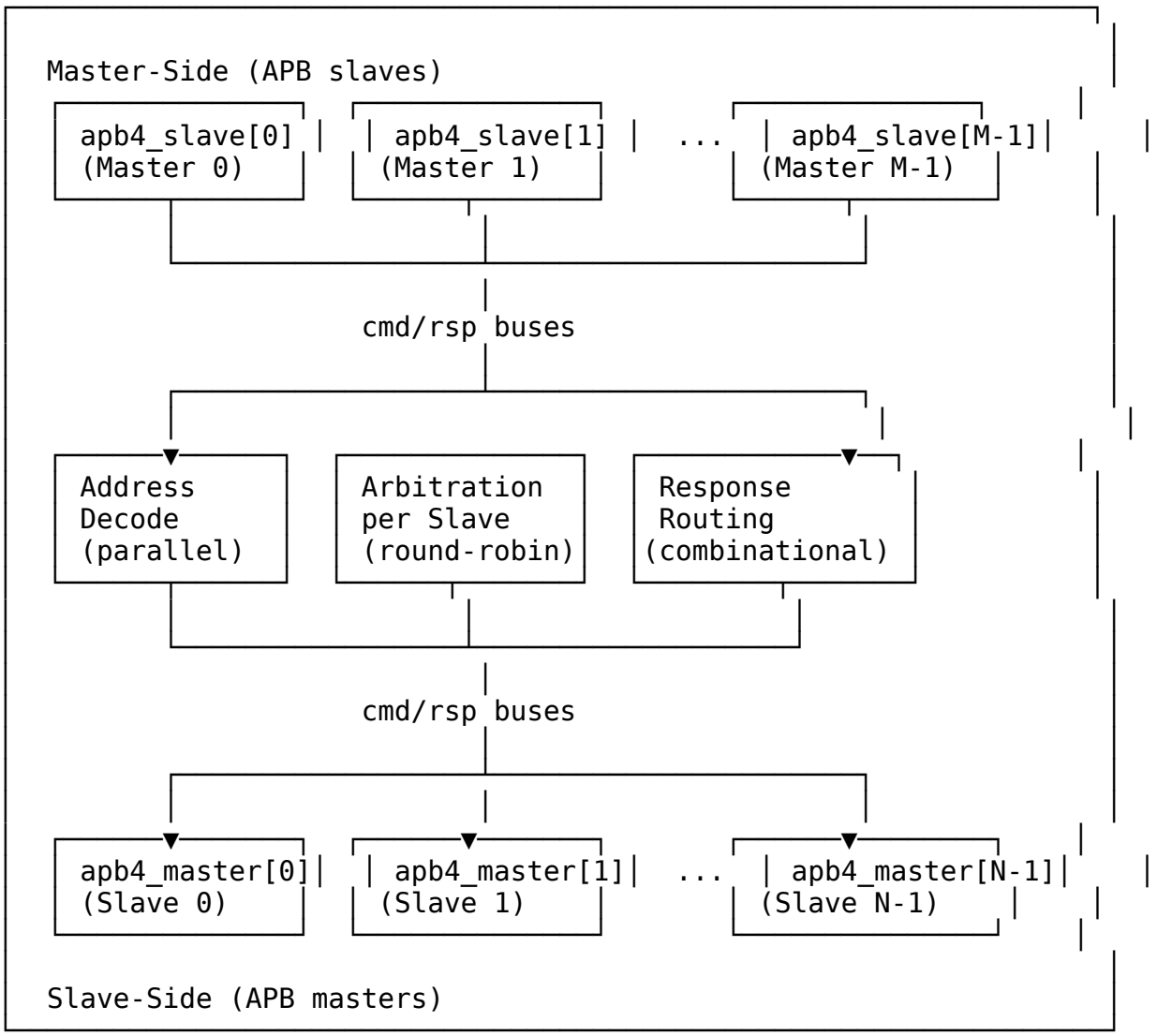
2.2 Design Trade-offs

Feature	Choice	Rationale
Arbitration	Round-robin per slave	Fair, simple, predictable
Address Map	64KB per slave	Sufficient for most peripherals
Grant Persistence	Hold through response	One transaction per grant; mask rotates after
Address Decode	Parallel decode	Low latency, simple logic

3. Architecture Overview

3.1 Top-Level Block Diagram

Lattice (M masters × N slaves)



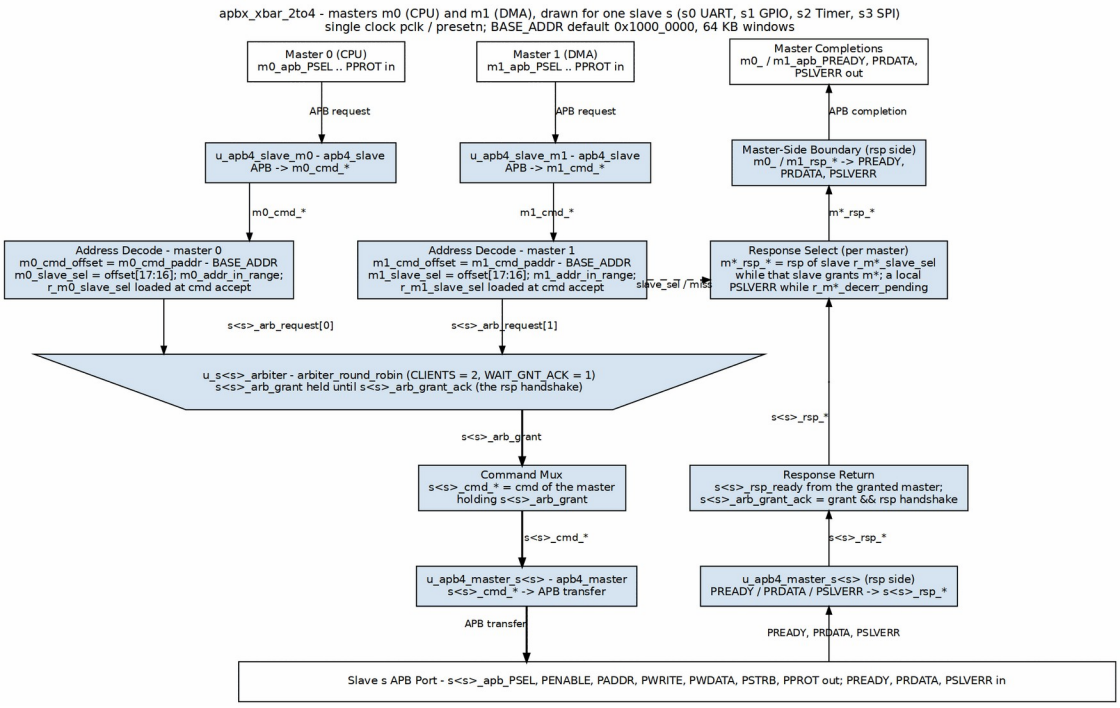


Figure: Lattice top-level architecture showing 2 masters connected to 4 slaves. Source: docs/lattice_mas/assets/graphviz/apbx_xbar_architecture.dot

3.2 Address Mapping

```
BASE_ADDR + 0x0000_0000 → 0x0000_FFFF : Slave 0 (64KB)
BASE_ADDR + 0x0001_0000 → 0x0001_FFFF : Slave 1 (64KB)
BASE_ADDR + 0x0002_0000 → 0x0002_FFFF : Slave 2 (64KB)
BASE_ADDR + 0x0003_0000 → 0x0003_FFFF : Slave 3 (64KB)
...
BASE_ADDR + 0x000F_0000 → 0x000F_FFFF : Slave 15 (64KB)
```

Default BASE_ADDR: 0x1000_0000

Address Decode Example:

The crossbar extracts the slave index from the upper bits of the address offset:

```
slave_index = (address - BASE_ADDR) >> 16  // Divide by 64KB (0x10000)
```

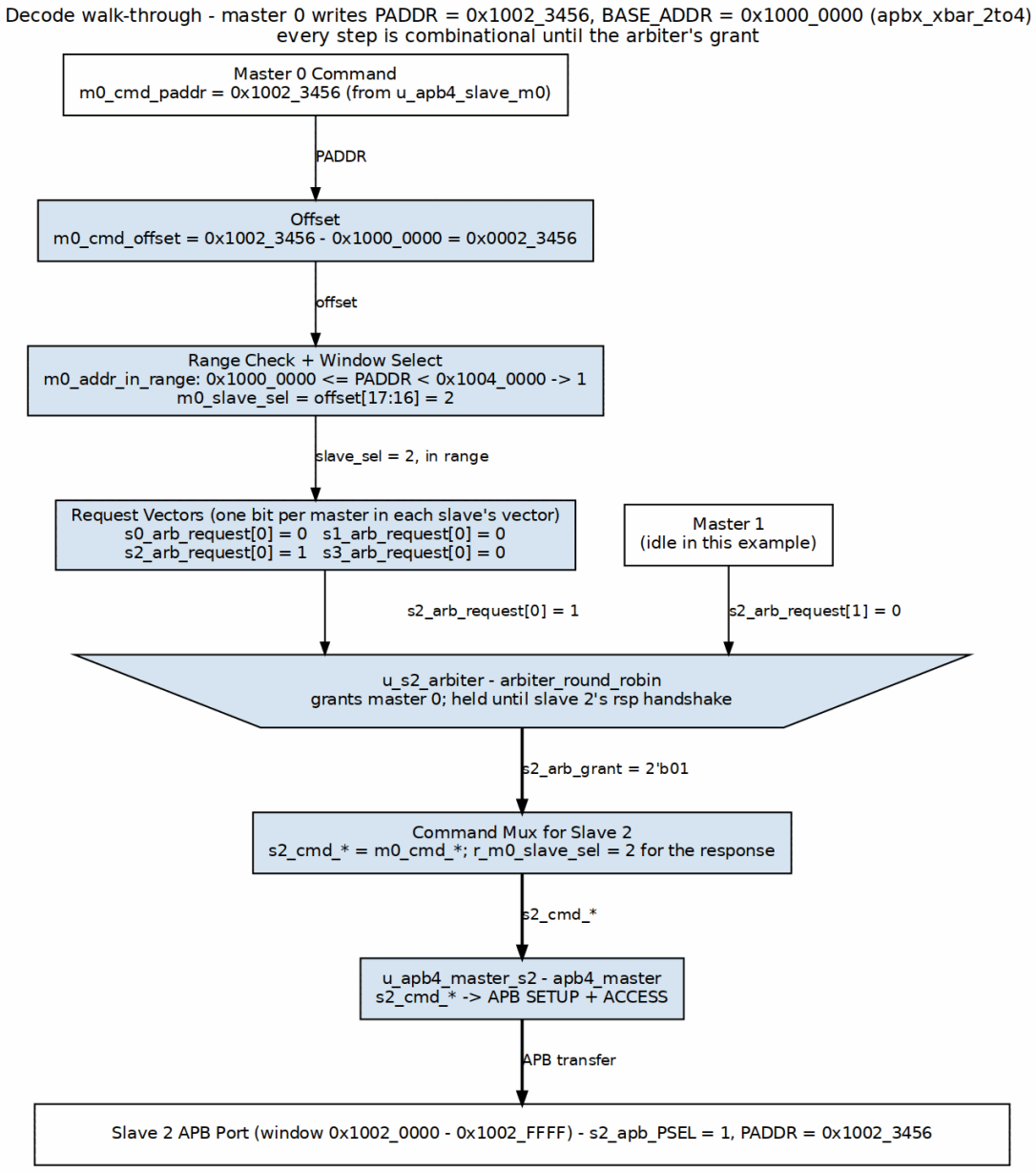


Figure: Address decode flow showing how address 0x10023456 routes to Slave 2. Source: docs/lattice_mas/assets/graphviz/address_decode_flow.dot

3.3 Arbitration Strategy

Per-Slave Round-Robin: - Each slave has independent arbiter - Master priority rotates after each grant - Grant held from command acceptance through response completion - No master can starve another master

Example:

```
Slave 0 accessed by M0, M1, M0 → Next grant goes to M1
Slave 1 accessed by M1, M1, M0 → Next grant goes to M1
  (the winner cannot immediately re-win: after an M0 grant the
   mask leaves only M1 eligible)
```

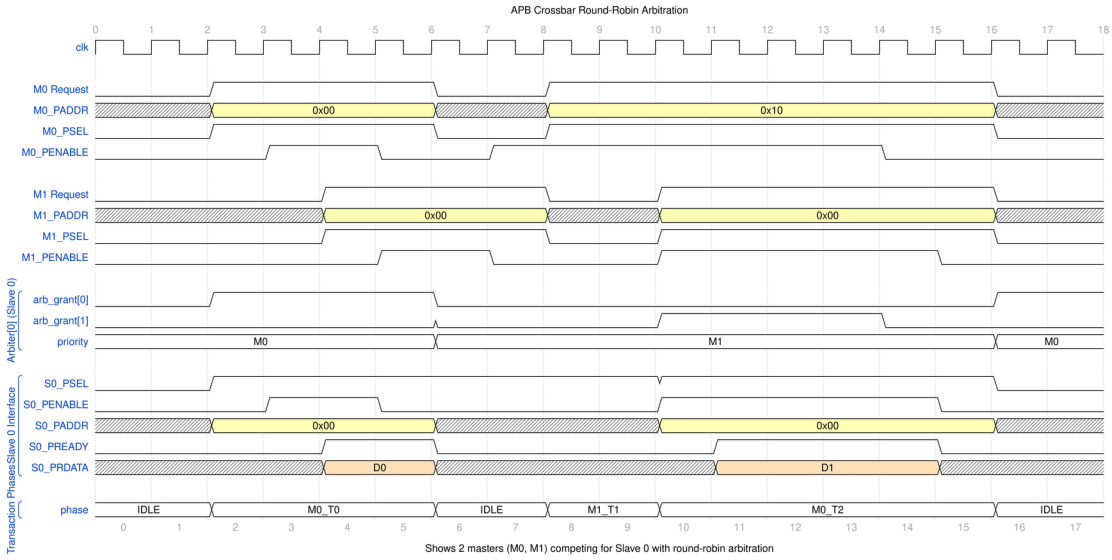


Figure: Round-robin arbitration timing showing 2 masters competing for Slave 0. Master priority rotates after each grant to ensure fair access. *Source: docs/lattice_mas/assets/wavedrom/arbitration_round_robin.json*

---

4. Available Variants

4.1 Pre-Generated Modules

Module	Masters	Slaves	Use Case
apbx_xbar_1to1	1	1	Protocol conversion, testing
apbx_xbar_2to1	2	1	Multi-master arbitration
apbx_xbar_1to4	1	4	Address decode, simple SoC
apbx_xbar_2to4	2	4	Full crossbar, typical SoC
apbx_xbar_2to2_mixed	2	2	Mixed APB4/APB5 (s0 is APB5)

LOC figures live in HAS ch05_performance/03_resources.md, which is the single source of truth for them. A duplicate column here disagreed with that table by up to 33%.

4.2 Wrapper Modules

These are TESTBENCH SCAFFOLDS, not integration building blocks. Each exposes `pclk` and `presetn` and nothing else — every APB signal is an internal net driven by the cocotb testbench. They exist because the four legacy tests drive a wrapper rather than the bare crossbar; a design cannot connect anything to one.

Wrapper	Configuration	Purpose
`apbx_xbar_1to1_wrap`	1×1	Test scaffold for `test_apbx_xbar_1to1.py`
`apbx_xbar_2to1_wrap`	2×1	Test scaffold for `test_apbx_xbar_2to1.py`
`apbx_xbar_1to4_wrap`	1×4	Test scaffold for `test_apbx_xbar_1to4.py`
`apbx_xbar_2to4_wrap`	2×4	Test scaffold for `test_apbx_xbar_2t`

Wrapper	Configuration	Purpose
		`o4.py`

To integrate, instantiate the crossbar itself. `apbx_xbar_2to2_mixed` has no wrapper precisely because its test drives the DUT directly, which is the better pattern.

---

5. Functional Requirements

FR-1: Arbitrary MxN Configuration

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Generate crossbar for any M masters and N slaves (up to 16x16)

Verification: Generator creates valid RTL for all tested configurations

FR-2: Round-Robin Arbitration

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Fair per-slave arbitration with rotating master priority

Verification: Arbitration stress tests (130+ transactions for 2to1)

FR-3: Address-Based Routing

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Automatic slave selection based on address ranges

Verification: Address decode validation (200+ transactions for 1to4)

FR-4: Back-to-Back Transactions

Priority: P1 (High) **Status:** Implemented and verified

Description: Consecutive transactions are accepted with no master-side idle cycles required. They do NOT overlap inside the fabric — `apb4_slave` is one-command-at-a-time. Sustained cadence is 10 pclk cycles PREADY-to-PREADY for a single-master variant and 11 for an arbitrated one, in each case one more than that variant's SETUP-to-PREADY latency of 9 or 10 (HAS 5.1/5.2).

Verification: Performance tests show consecutive transactions

FR-5: Configurable Address Map

Priority: P1 (High) **Status:** Implemented and verified

Description: BASE_ADDR parameter sets base of address map

Verification: Tests with multiple BASE_ADDR values

---

6. Interface Specifications

6.1 Master-Side Interface (APB Slave)

Per-master APB slave interface:

```
input  logic                    m<i>_apb_PSEL
input  logic                    m<i>_apb_PENABLE
input  logic [ADDR_WIDTH-1:0]   m<i>_apb_PADDR
input  logic                    m<i>_apb_PWRITE
input  logic [DATA_WIDTH-1:0]   m<i>_apb_PWDATA
input  logic [STRB_WIDTH-1:0]   m<i>_apb_PSTRB
input  logic [2:0]              m<i>_apb_PPROT
output logic [DATA_WIDTH-1:0]   m<i>_apb_PRDATA
output logic                    m<i>_apb_PSLVERR
output logic                    m<i>_apb_PREADY
```

6.2 Slave-Side Interface (APB Master)

Per-slave APB master interface:

```
output logic                    s<j>_apb_PSEL
output logic                    s<j>_apb_PENABLE
output logic [ADDR_WIDTH-1:0]   s<j>_apb_PADDR
output logic                    s<j>_apb_PWRITE
output logic [DATA_WIDTH-1:0]   s<j>_apb_PWDATA
output logic [STRB_WIDTH-1:0]   s<j>_apb_PSTRB
output logic [2:0]              s<j>_apb_PPROT
input  logic [DATA_WIDTH-1:0]   s<j>_apb_PRDATA
input  logic                    s<j>_apb_PSLVERR
input  logic                    s<j>_apb_PREADY
```

---

7. Parameter Configuration

Parameter	Type	Default	Range	Description
ADDR_WIDTH	int	32	18-64 for multi-slave variants (the decode part-select needs bit 17); 1-64 for 1to1/2to1	Address bus width
DATA_WIDTH	int	32	8,16,32,64	Data bus width
STRB_WIDTH	int	DATA_WIDTH/8	-	Strobe width (auto-calc)
BASE_ADDR	logic [ADDR_WIDTH-1:0]	0x10000000	Any except the top S x 64KB, where BASE_ADDR + span wraps and the range check can never pass	Base address for slave map. Inert on 1to1/2to1 (no decode)

8. Generator Usage

8.1 Pre-Generated Variants

Use existing modules directly:

```
# Available in
projects/components/fabric-gen-ip/lattice-apbx-xbar/rtl/
apbx_xbar_1to1.sv
apbx_xbar_2to1.sv
apbx_xbar_1to4.sv
apbx_xbar_2to4.sv
apbx_xbar_2to2_mixed.sv
```

8.2 Custom Generation

Generate custom MxN crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6
python generate_xbars.py --masters 4 --slaves 8 --base-addr 0x80000000
```

8.3 Generator Options

```
python generate_xbars.py --help

Options:
  --masters M       Number of masters (1-16)
  --slaves N        Number of slaves (1-16)
  --base-addr ADDR  Base address (default: 0x10000000)
```

---

9. Performance Characteristics

9.1 Latency

Path	M = 1	M > 1	Notes
Command path	6	7	master SETUP (1) + forward path (5, or 6 with a registered arbiter grant)
Response path	3	3	arbitration is on the command side only
Total	**9**	**10**	SETUP->PREADY, uncontended, zero-wait slave (8 / 9 ACCESS->PREADY)

M = 1 is `1to1` and `1to4`, which the generator emits with no arbiter at all. M > 1 is `2to1`, `2to4` and `2to2_mixed` — including the 2x4 this document calls the typical SoC case. This table gave only the M = 1 figures, unlabelled, until 2026-08-30.

APB's 2-cycle minimum applies to a directly-attached slave, not through this fabric: the crossbar converts APB→cmd/rsp→APB across registered skid buffers in both directions. See HAS 5.2 for the cycle-by-cycle breakdown and the measurement.

9.2 Throughput

- **Back-to-back transactions:** Supported, but not overlapped — `apb4_slave` is a one-command-at-a-time FSM, so the next command is captured only after the previous transaction completes
- **Maximum rate:** ~1 transaction per 10 pclk cycles (1to1, 1to4) or 11 (2to1, 2to4, 2to2_mixed) per master (measured back-to-back at an always-ready slave)

9.3 Resource Utilization (Estimated)

See HAS ch05_performance/03_resources.md — that table is the single source of truth. An earlier duplicate here quoted figures 2-3x smaller in every cell; both were unsourced hand estimates, so the duplicate is withdrawn rather than reconciled.

10. Verification Status

10.1 Test Coverage

Test	Transactions	Status	Coverage
test_apbx_xbar_1to1	100+	Pass	Basic connectivity
test_apbx_xbar_2to1	130+	Pass	Arbitration stress
test_apbx_xbar_1to4	200+	Pass	Address decode
test_apbx_xbar_2to4	350+	Pass	Full crossbar stress
test_apbx_xbar_2to2_mixed	8 directed	Pass	All four APB4/APB5 pairings, sideband gating, decode miss, structural pin check
test_apbx_xbar_timing	2 variants	Pass	Published latency/cadence figures, both variant classes

Overall: 100% passing (9/9), >750 transactions plus the directed mixed-version and timing checks. `2to2_mixed` also has a formal harness (`formal/apbx_xbar/apbx_xbar_2to2_mixed/`).

10.2 Test Methodology

Test Structure: - CocoTB framework - Random transaction generation - Variable delay profiles - Address range validation - Arbitration fairness checks

Verification Points: - Correct address routing - Round-robin arbitration - Response integrity - Back-to-back transactions - Error propagation

---

11. Integration Guide

11.1 Simple Integration (1to4)

```
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master interface
    .m0_apb_PSEL(cpu_psel),
    .m0_apb_PENABLE(cpu_penable),
    .m0_apb_PADDR(cpu_paddr),
    .m0_apb_PWRITE(cpu_pwrite),
    .m0_apb_PWDATA(cpu_pwdata),
    .m0_apb_PSTRB(cpu_pstrb),
    .m0_apb_PPROT(cpu_pprot),
    .m0_apb_PRDATA(cpu_prdata),
    .m0_apb_PSLVERR(cpu_pslverr),
    .m0_apb_PREADY(cpu_pready),

    // Slave 0: UART (0x1000_0000 - 0x1000_FFFF)
    .s0_apb_PSEL(uart_psel),
    .s0_apb_PENABLE(uart_penable),
    // ... (similar connections)

    // Slave 1: GPIO (0x1001_0000 - 0x1001_FFFF)
    .s1_apb_PSEL(gpio_psel),
    // ... (similar connections)

    // Slave 2: Timer (0x1002_0000 - 0x1002_FFFF)
    // Slave 3: SPI (0x1003_0000 - 0x1003_FFFF)
);
```

11.2 Multi-Master Integration (2to4)

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
```

```
    .m0_apb_PSEL(cpu_psel),
    // ... (full interface)

    // Master 1: DMA
    .m1_apb_PSEL(dma_psel),
    // ... (full interface)

    // Slaves 0-3: Peripherals
    // ... (slave connections)
);
```

---

12. Design Constraints

12.1 Limitations

Constraint	Value	Rationale
Max masters	16	Generator limit (configurable)
Max slaves	16	Generator limit (configurable)
Slave region size	64KB in every shipped variant	Baked in at generation (`--slave-size`, default 4KB); not overridable at elaboration
No slave disable	N/A	All slaves always active
No timeout	N/A	Assumes slaves always respond

12.2 Assumptions

1. **Slave responses:** All slaves respond eventually (no timeout handling)
2. **Address ranges:** Slaves occupy 64KB regions starting at BASE_ADDR
3. **APB compliance:** All masters and slaves follow APB protocol
4. **Single clock domain:** All signals synchronous to pclk

13. Future Enhancements

13.1 Potential Features

- **Configurable slave region sizes:** Not just 64KB
- **Slave enable/disable:** Dynamic slave activation
- **Timeout detection:** Watchdog for hung slaves
- **Transaction ordering:** Optional in-order completion
- **Priority arbitration:** Weighted instead of round-robin

13.2 Generator Improvements

- **HDL output formats:** Support Verilog, VHDL, Chisel
- **Automated testing:** Generate tests alongside RTL
- **Documentation generation:** Auto-generate integration guides
- **Synthesis scripts:** Generate vendor-specific scripts

---

14. References

14.1 Internal Documentation

- `README.md` - Quick start guide
- `CLAUDE.md` - AI assistant guidelines
- `bin/generate_xbars.py` - Generator script
- `dv/tests/` - Test implementations

14.2 External Standards

- **AMBA APB Protocol v2.0** - ARM IHI 0024C
- **APB4 Extensions** - AMBA 5 specification

14.3 Related Components

- `rtl/amba/apb4/apb4_slave.sv` - Master-side building block
- `rtl/amba/apb4/apb4_master.sv` - Slave-side building block
- `projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/apbx_xbar_generator.py` - Main generator

Document Version: 1.0 **Last Review:** 2025-10-19 **Next Review:** 2026-01-01 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Claude Code Guide: Lattice Component

Version: 1.0 **Last Updated:** 2025-10-19 **Purpose:** AI-specific guidance for working with Lattice component

Codename: lattice · **Directory:** `lattice-apbx-xbar` (fabric-gen-ip family with `bridge` and `delta`); RTL modules keep the `apbx_xbar_*` prefix

---

Quick Context

What: Lattice Generator - Parametric APB interconnect for connecting M masters to N slaves
Status: Production Ready (all tests passing) **Your Role:** Help users generate, integrate, and customize APB crossbars

Complete Specification:
`projects/components/fabric-gen-ip/lattice-apbx-xbar/PRD.md` ← **Always reference this for technical details**

Critical Rules for This Component

Rule #0: This is a GENERATOR, Not Just Modules

IMPORTANT: Lattice is both: 1. **Pre-generated modules** (1to1, 2to1, 1to4, 2to4, 2to2_mixed) in `rtl/` 2. **Python generator** (`bin/generate_xbars.py`) for custom configurations

When users ask for crossbar: - Check if pre-generated variant exists first - Only suggest generation if custom size needed

Decision Tree:

```
User needs crossbar?
├─ 1x1, 2x1, 1x4, 2x4? → Use pre-generated module
├─ Mixed APB4/APB5 2x2? → Use apbx_xbar_2to2_mixed
└─ Custom MxN? → Run generator script
```

Rule #1: Address Map is Fixed Per-Slave

Each slave occupies 64KB region:

```
Slave 0: BASE_ADDR + 0x0000_0000 → 0x0000_FFFF
Slave 1: BASE_ADDR + 0x0001_0000 → 0x0001_FFFF
Slave 2: BASE_ADDR + 0x0002_0000 → 0x0002_FFFF
...
```

The window size is not a module parameter, but it IS a generator option. Every *shipped* variant uses 64KB windows because `bin/generate_xbars.py` passes `slave_size=0x10000`. The generator's own default is 4KB, and `apbx_xbar_generator.py --slave-size` accepts any power of 2 >= 256 bytes. Once generated the size is baked into the decode part-select, so it cannot be overridden at elaboration — only `BASE_ADDR` can, and only on the decoding variants.

If user asks for different sizes:

```
WRONG: "Let me modify the generator to support custom sizes per slave"

CORRECT: "The shipped variants use 64KB per slave. You can:
1. Use BASE_ADDR to shift the whole map (decoding variants only)
2. For a different window, regenerate with --slave-size (power of 2,
>= 256B)
3. Or use multiple crossbars with different BASE_ADDR values"
```

Rule #2: Know the Pre-Generated Variants

Available in `rtl/` directory:

Module	M×N	Use Case	When to Recommend
apbx_xbar_1to1	1×1	Passthrough	Protocol conversion, testing
apbx_xbar_2to1	2×1	Arbitration	Multi-master to single peripheral
apbx_xbar_1to4	1×4	Decode	Single CPU to multiple peripherals
apbx_xbar_2to4	2×4	Full crossbar	CPU + DMA to peripherals
apbx_xbar_2to2_mixed	2×2	Mixed APB4/APB5	APB5 peripheral alongside APB4

Always suggest pre-generated first!

Rule #3: Generator Syntax

Generate custom crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6
```

Options:

```
--masters M         Number of masters (1-16)
--slaves N          Number of slaves (1-16)
--base-addr ADDR    Base address (default: 0x10000000)
```

Example:

```
# Generate 3x8 crossbar with custom base
python generate_xbars.py --masters 3 --slaves 8 --base-addr 0x80000000

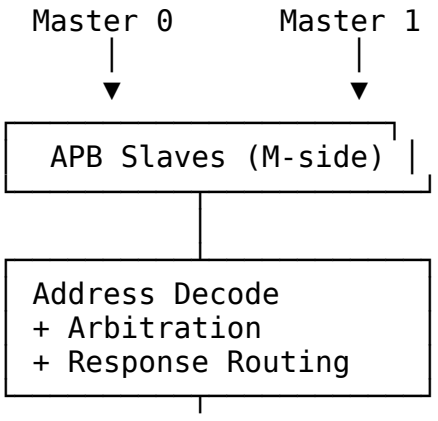
# Generate a 5x5 variant
python generate_xbars.py --masters 5 --slaves 5
```

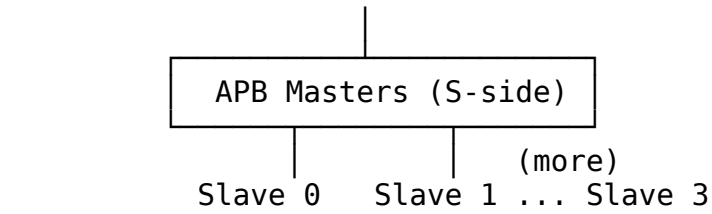
Architecture Quick Reference

Module Organization

```
projects/components/fabric-gen-ip/lattice-apbx-xbar/
├── rtl/
│   ├── apbx_xbar_1to1.sv        Core crossbar modules
│   ├── apbx_xbar_2to1.sv
│   ├── apbx_xbar_1to4.sv
│   ├── apbx_xbar_2to4.sv
│   ├── apbx_xbar_2to2_mixed.sv  (m0=APB4, m1=APB5, s0=APB5, s1=APB4)
│   └── wrappers/               testbench scaffolds, not integration
wrappers
│       ├── apbx_xbar_1to1_wrap.sv
│       ├── apbx_xbar_2to1_wrap.sv
│       ├── apbx_xbar_1to4_wrap.sv
│       └── apbx_xbar_2to4_wrap.sv
├── bin/
│   └── generate_xbars.py       Generator script
├── dv/
│   └── tests/
│       ├── test_apbx_xbar_1to1.py
│       ├── test_apbx_xbar_2to1.py
│       ├── test_apbx_xbar_1to4.py
│       ├── test_apbx_xbar_2to4.py
│       ├── test_apbx_xbar_2to2_mixed.py
│       ├── test_apbx_xbar_timing.py     published latency/cadence
figures
│       ├── test_apbx_xbar_parity.py     generates a parity-on all-
APB5 2x2 (APB5 *CHK, PWAKEUP)
│       └── GTKW/                Waveform configs
├── PRD.md                       Complete specification
├── CLAUDE.md                    This file
└── README.md                    Quick start guide
```

Block Diagram (2×4 Example)




APB Masters (S-side)
Slave 0
Slave 1
(more)
... Slave 3

Common User Questions and Responses

Q: “How do I connect a CPU to 4 peripherals?”

A: Use the pre-generated 1to4 crossbar:

```
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)  // Start of peripheral space
) u_peripheral_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // CPU connection (master 0)
    .m0_apb_PSEL(cpu_psel),
    .m0_apb_PENABLE(cpu_penable),
    .m0_apb_PADDR(cpu_paddr),
    .m0_apb_PWRITE(cpu_pwrite),
    .m0_apb_PWDATA(cpu_pwdata),
    .m0_apb_PSTRB(cpu_pstrb),
    .m0_apb_PPROT(cpu_pprot),
    .m0_apb_PRDATA(cpu_prdata),
    .m0_apb_PSLVERR(cpu_pslverr),
    .m0_apb_PREADY(cpu_pready),

    // Peripheral 0: UART @ 0x1000_0000
    .s0_apb_PSEL(uart_psel),
    .s0_apb_PENABLE(uart_penable),
    // ... (full interface)

    // Peripheral 1: GPIO @ 0x1001_0000
    // Peripheral 2: Timer @ 0x1002_0000
    // Peripheral 3: SPI @ 0x1003_0000
);
```

Address map: - UART: 0x1000_0000 - 0x1000_FFFF - GPIO: 0x1001_0000 - 0x1001_FFFF - Timer: 0x1002_0000 - 0x1002_FFFF - SPI: 0x1003_0000 - 0x1003_FFFF

See: PRD.md Section 11.1

Q: “I need CPU and DMA to access peripherals. Which crossbar?”

A: Use the 2to4 crossbar:

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
```

```
    .BASE_ADDR(32'h1000_0000)
) u_soc_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
    .m0_apb_PSEL(cpu_psel),
    // ... (full interface)

    // Master 1: DMA
    .m1_apb_PSEL(dma_psel),
    // ... (full interface)

    // Slaves 0-3: Peripherals
    // ... (slave connections)
);
```

Key feature: Round-robin arbitration per slave ensures fair access between CPU and DMA.

See: PRD.md Section 11.2

Q: "What if I need 3 masters and 8 slaves?"

A: Generate custom crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 8   # writes
apbx_xbar_3to8.sv next to the script

# Check generated file, then move it into rtl/
ls -lh apbx_xbar_3to8.sv
mv apbx_xbar_3to8.sv ../rtl/
```

Then instantiate like any other crossbar:

```
apbx_xbar_3to8 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_custom_xbar (
    // ... connections for 3 masters and 8 slaves
);
```

See: PRD.md Section 8

Q: "How does arbitration work?"

A: Per-slave round-robin:

Each slave has its own arbiter that rotates master priority:

```
Example with 2 masters accessing Slave 0:

Transaction 1: M0 requests → M0 granted
Transaction 2: M0 and M1 request → M1 granted (rotated)
Transaction 3: M1 requests → M1 granted
Transaction 4: M0 and M1 request → M0 granted (rotated)
```

Key points: - **Independent per slave:** Each slave arbitrates independently - **Fair:** No master can starve another - **Grant persistence:** Once granted, master holds slave until response completes - **Back-to-back:** supported, but not overlapped — 10 pclk cycles per transfer for 1to1/1to4, 11 for the arbitrated variants (2to1, 2to4, 2to2_mixed), PREADY-to-PREADY (see HAS 5.1/5.2)

See: PRD.md Section 3.3

Q: "Can I change the address map?"

A: BASE_ADDR parameter shifts entire map, but per-slave size is fixed:

```
// Default: 0x1000_0000
apbx_xbar_1to4 #(.BASE_ADDR(32'h1000_0000)) u_xbar1 (...);
// Slaves at: 0x1000_0000, 0x1001_0000, 0x1002_0000, 0x1003_0000

// Shifted: 0x8000_0000
apbx_xbar_1to4 #(.BASE_ADDR(32'h8000_0000)) u_xbar2 (...);
// Slaves at: 0x8000_0000, 0x8001_0000, 0x8002_0000, 0x8003_0000
```

Limitation: Each slave always occupies 64KB (0x10000 bytes)

Workaround for different sizes: 1. Use multiple crossbars with different BASE_ADDR 2. Modify generator's `addr_offset` calculation 3. Use address masking in slaves

See: PRD.md Section 3.2

Q: "How do I run tests?"

A: Use the area Makefile (never bare pytest):

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/

# Run specific test
make run-apbx_xbar_1to4-gate

# Run all tests -- clean-all first for a run you intend to trust
make clean-all && make run-all-gate

# Run with waveforms (WAVES=1 via the target, not --vcd)
```

```
make run-apbx_xbar_2to4-gate-waves
# create_view_cmd() writes the gtkwave command beside the log
```

Test coverage: - **test_apbx_xbar_1to1**: 100+ transactions (passthrough) - **test_apbx_xbar_2to1**: 130+ transactions (arbitration) - **test_apbx_xbar_1to4**: 200+ transactions (address decode) - **test_apbx_xbar_2to4**: 350+ transactions (full stress) - **test_apbx_xbar_2to2_mixed**: 8 directed — all four APB4/APB5 pairings, sideband gating, decode miss, structural pin check; plus `_pwakeup`: PWAKEUP forwarded requester -> completer (IHI 0024E 3.7.2) - **test_apbx_xbar_parity**: no shipped variant enables parity, so this generates one into its sim_build and checks odd `*CHK` parity (IHI 0024E ch. 5), error injection, and PWAKEUP forwarding - **test_apbx_xbar_timing**: both variant classes — asserts the published latency, cadence and per-path breakdown so the numbers cannot drift

All tests passing (22/22 at func, 2026-10-12) — and `2to2_mixed` additionally has a formal harness at `formal/apbx_xbar/apbx_xbar_2to2_mixed/`.

See: `PRD.md` Section 10

Integration Patterns

Pattern 1: Simple Peripheral Interconnect

```
// CPU to 4 peripherals
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_periph_xbar (
    .pclk(sys_clk),
    .presetn(sys_rst_n),

    // CPU side
    .m0_apb_PSEL(cpu_apb_psel),
    .m0_apb_PENABLE(cpu_apb_penable),
    .m0_apb_PADDR(cpu_apb_paddr),
    .m0_apb_PWRITE(cpu_apb_pwrite),
    .m0_apb_PWDATA(cpu_apb_pwdata),
    .m0_apb_PSTRB(cpu_apb_pstrb),
    .m0_apb_PPROT(cpu_apb_pprot),
    .m0_apb_PRDATA(cpu_apb_prdata),
    .m0_apb_PSLVERR(cpu_apb_pslverr),
    .m0_apb_PREADY(cpu_apb_pready),

    // Peripherals
    .s0_apb_PSEL(uart_psel), /* ... full interface ... */
    .s1_apb_PSEL(gpio_psel), /* ... */
    .s2_apb_PSEL(timer_psel), /* ... */
    .s3_apb_PSEL(spi_psel) /* ... */
);
```

Pattern 2: Multi-Master System

```
// CPU + DMA to peripherals
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h4000_0000)
) u_soc_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
    .m0_apb_PSEL(cpu_psel),
    /* ... full CPU interface ... */

    // Master 1: DMA Controller
```

```
    .m1_apb_PSEL(dma_psel),
    /* ... full DMA interface ... */

    // Slave 0-3: Memory-mapped peripherals
    .s0_apb_PSEL(mem_ctrl_psel), /* ... */
    .s1_apb_PSEL(uart_psel), /* ... */
    .s2_apb_PSEL(i2c_psel), /* ... */
    .s3_apb_PSEL(adc_psel) /* ... */
);
```

Pattern 3: Hierarchical Interconnect

```
// Top-level crossbar
apbx_xbar_1to4 u_top_xbar (
    // .m0_apb_PSEL(cpu_apb_PSEL), .m0_apb_PENABLE(...), ...  (elided)
    .s0_apb_PSEL(periph_bus0_PSEL), /* ...rest of the port... */  //
To sub-crossbar 0
    .s1_apb_PSEL(periph_bus1_PSEL), /* ...rest of the port... */  //
To sub-crossbar 1
    .s2_apb_PSEL(mem_ctrl_PSEL), /* ...rest of the port... */     //
Direct to memory controller
    .s3_apb_PSEL(dma_ctrl_PSEL), /* ...rest of the port... */      //
Direct to DMA
);

// Sub-crossbar 0 for low-speed peripherals
apbx_xbar_1to4 u_periph_xbar0 (
    .m0_apb_PSEL(periph_bus0_PSEL), /* ...rest of the port... */
    .s0_apb_PSEL(uart0_PSEL), /* ...rest of the port... */
    .s1_apb_PSEL(gpio0_PSEL), /* ...rest of the port... */
    .s2_apb_PSEL(i2c0_PSEL), /* ...rest of the port... */
    .s3_apb_PSEL(spi0_PSEL), /* ...rest of the port... */
);

// Sub-crossbar 1 for high-speed peripherals
apbx_xbar_1to4 u_periph_xbar1 (
    .m0_apb_PSEL(periph_bus1_PSEL), /* ...rest of the port... */
    .s0_apb_PSEL(uart1_PSEL), /* ...rest of the port... */
    .s1_apb_PSEL(timer_PSEL), /* ...rest of the port... */
    .s2_apb_PSEL(pwm_PSEL), /* ...rest of the port... */
    .s3_apb_PSEL(adc_PSEL), /* ...rest of the port... */
);
```

Anti-Patterns to Catch

Anti-Pattern 1: Generating When Pre-Generated Exists

```
WRONG:
User: "I need a 2×4 crossbar"
You: "Let me generate that for you..."
python generate_xbars.py --masters 2 --slaves 4

CORRECTED:
"Use the pre-generated apbx_xbar_2to4.sv in the rtl/ directory.
No generation needed!"
```

Anti-Pattern 2: Assuming Custom Per-Slave Sizes

```
WRONG:
User: "Can I make slave 0 256KB and slave 1 4KB?"
You: "Sure, let me modify the parameters..."

CORRECTED:
"The shipped variants use 64KB per slave. For a different window:
1. Regenerate with --slave-size (power of 2, >= 256B)
2. Or use multiple crossbars with different BASE_ADDR
3. Or implement address masking in slaves"
```

Anti-Pattern 3: Not Mentioning Address Map

```
WRONG:
User: "How do I integrate the crossbar?"
You: *Shows port connections only*

CORRECTED:
"Here's the integration with address map:
apbx_xbar_1to4 #(.BASE_ADDR(32'h1000_0000)) u_xbar (...);

Address map:
- Slave 0: 0x1000_0000 - 0x1000_FFFF
- Slave 1: 0x1001_0000 - 0x1001_FFFF
- Slave 2: 0x1002_0000 - 0x1002_FFFF
- Slave 3: 0x1003_0000 - 0x1003_FFFF"
```

Anti-Pattern 4: Forgetting About Wrappers

```
WRONG:
User: "I need a quick 10×10 crossbar"
You: "Run the generator with --masters 10 --slaves 10"

CORRECTED:
"We have pre-configured wrappers in rtl/wrappers/:
```

```
- apbx_xbar_1to1_wrap.sv, apbx_xbar_2to1_wrap.sv,
  apbx_xbar_1to4_wrap.sv, apbx_xbar_2to4_wrap.sv

Those four are TESTBENCH SCAFFOLDS, not integration wrappers: each has
only
pclk and presetn, with every APB signal internal to the cocotb bench,
so a
design cannot connect to one. Instantiate the crossbar directly. For
any
other MxS, generate one with bin/generate_xbars.py -- and there is no
m10_s10 wrapper in the tree."
```

Debugging Workflow

Issue: Address Not Routing Correctly

Check in order: 1. BASE_ADDR parameter set correctly? 2. Address within slave's 64KB region? 3. PSEL signal asserted by master? 4. All APB signals properly connected?

Calculate expected slave:

```
slave_index = (address - BASE_ADDR) >> 16  // Divide by 64KB
```

Debug commands:

```
cd dv/tests && make run-apbx_xbar_1to4-gate-waves
# gtkwave command is written beside the log by create_view_cmd()
```

Issue: Arbitration Not Fair

Symptoms: - One master dominates - Other masters starved

Check: 1. Arbiters instantiated per slave? 2. Round-robin logic correct? 3. Grant persistence working?

Verify with tests:

```
cd dv/tests && make run-apbx_xbar_2to1-gate   # Arbitration stress
test
```

Issue: Back-to-Back Transactions Stalling

Check: 1. Grant persistence enabled? 2. Slaves responding with PREADY? 3. No unintended pipeline bubbles?

View waveforms:

```
cd dv/tests && make run-apbx_xbar_2to4-gate-waves
# gtkwave command is written beside the log by create_view_cmd()
```

Quick Commands

```
# List available crossbars
ls projects/components/fabric-gen-ip/lattice-apbx-xbar/rtl/*.sv

# Generate custom crossbar
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6

# Run all tests
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests
make clean-all && make run-all-gate

# Run specific test with waveforms
make run-apbx_xbar_2to4-gate-waves

# View waveforms: create_view_cmd() writes a ready-made command
# beside the log; no hand-built gtkwave path needed

# Check documentation
cat projects/components/fabric-gen-ip/lattice-apbx-xbar/PRD.md
cat projects/components/fabric-gen-ip/lattice-apbx-xbar/README.md
```

Remember

1. **Check pre-generated first** - Don't generate unnecessarily
2. **Address map matters** - Always mention BASE_ADDR + 64KB regions
3. **Fair arbitration** - Round-robin per slave
4. **Complete connections** - All APB signals must be wired
5. **Tests available** - 100% passing, comprehensive coverage
6. **Wrappers are TEST SCAFFOLDS** - `rtl/wrappers/` carries only pclk/presetn; instantiate the crossbar directly (see Anti-Pattern 4)
7. **Generator limits** - Up to 16×16 (configurable)

Version: 1.0 **Last Updated:** 2025-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Lattice — APB Crossbar

Codename: lattice · **Directory:** `lattice-apbx-xbar` — the fabric-gen-ip family alongside `bridge` and `delta`. RTL modules and filelists keep the `apbx_xbar_*` prefix; “lattice” is the documentation and prose name.

This directory contains the APB crossbar family: a parameterized core plus generated fixed-configuration variants. Every port independently speaks **APB4 or APB5**, so one fabric can carry a mix of both.

The `x` in `apbx` is the version wildcard — it is not a separate protocol.

Architecture

Generated variants follow a consistent design:

```
Master Side:     apb4_slave / apb5_slave convert APB → cmd/rsp
interface
Internal:        Round-robin arbitration + address decoding
Slave Side:      apb4_master / apb5_master convert cmd/rsp → APB
interface
```

Every variant in this family is generator output built on that structure; there is no hand-written alternative core.

Key Features

- **Independent arbitration per slave**: Each slave has its own round-robin arbiter
- **Grant persistence**: Grants held from command acceptance through response completion
- **Address decoding**: Automatic slave selection based on address ranges
- **Mixed APB4/APB5**: per-port version selection, sideband gated at both ends
- **Proven architecture**: Uses production-tested apb4/apb5 slave and master modules

Why mixing needs no converters

APB5 is APB4's transfer protocol plus extra sideband pins — the handshake and phases are identical. So sideband rides the same select/grant/demux muxes as `PADDR`, and the version selection only gates *contribution*: an APB4 master contributes `'0`, and an APB4 slave is never driven with nonzero sideband. Both gates are compile-time constants, so synthesis prunes any pairing that cannot occur, and an all-APB4 build is bit-identical to the pre-APB5 crossbar.

`PWAKEUP` is not sideband in this sense. It is a Requester output (AMBA APB IHI 0024E section 3.7.2). It is an input on an APB5 master port and an output on an APB5 slave port. Each APB5 slave port drives the OR of its boundary IP's own pending/in-flight wake and every APB5 master's `PWAKEUP`, so even an APB4 master's transfer arrives with `PWAKEUP` high. See the GAS chapter *Mixed APB4/APB5 and Parity*.

Available Modules

Module	Description	Masters	Slaves	Use Case
`apbx_xbar_1to1.sv`	Simple passthrough	1	1	Protocol conversion, testing
`apbx_xbar_2to1.sv`	Arbitration only	2	1	Multi-master to single slave
`apbx_xbar_1to4.sv`	Address decode only	1	4	Single master to multi-slave
`apbx_xbar_2to4.sv`	Full crossbar	2	4	Multi-master to multi-slave
`apbx_xbar_2to2_mixed.sv`	Mixed APB4/APB5	2	2	m0=APB4, m1=APB5, s0=APB5, s1=APB4

Generating Crossbars

Method 1: Convenience Script (Recommended)

Generate all standard variants:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py
```

Generate custom variant:

```
python generate_xbars.py --masters 3 --slaves 6
python generate_xbars.py --masters 4 --slaves 8 --base-addr 0x80000000
```

Generate a mixed-version variant — versions are positional per port, and are baked in at generation time rather than being a runtime parameter:

```
# in generate_xbars.py
mixed = [
    ((2, 2), ['apb4', 'apb5'], ['apb5', 'apb4'], '_mixed'),
]
```

`rtl/*.sv` is generator output. The generator emits the final form (SPDX banner, `reset_defs.svh` include, reset macros) straight into `rtl/`, so there is no post-processing step and nothing to hand-edit. Regeneration is idempotent:

```
python3 generate_xbars.py && git status --short ../rtl/   # expect no
output
```

Method 2: Direct Generator

Use the main generator for more control:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python apbx_xbar_generator.py --masters 2 --slaves 4 --output
../rtl/apbx_xbar_2to4.sv
```

Address Map

The crossbars use a uniform address map, configurable via the BASE_ADDR parameter. The slave index is decoded from the OFFSET (PADDR - BASE_ADDR), so BASE_ADDR needs no span alignment (the one illegal region is the top S x 64KB of the address space, where BASE_ADDR + S*64KB wraps 32-bit and the range check can never pass).

```
Slave 0: [BASE_ADDR + 0x0000_0000, BASE_ADDR + 0x0000_FFFF]  (64KB)
Slave 1: [BASE_ADDR + 0x0001_0000, BASE_ADDR + 0x0001_FFFF]  (64KB)
Slave 2: [BASE_ADDR + 0x0002_0000, BASE_ADDR + 0x0002_FFFF]  (64KB)
Slave 3: [BASE_ADDR + 0x0003_0000, BASE_ADDR + 0x0003_FFFF]  (64KB)
...
```

Default BASE_ADDR: `0x1000_0000`

Parameters

The GENERATED modules support these parameters:

Parameter	Default	Description
ADDR_WIDTH	32	Address bus width
DATA_WIDTH	32	Data bus width
STRB_WIDTH	DATA_WIDTH/8	Strobe width (auto-calculated)
BASE_ADDR	0x10000000	Base of the slave address map

Per-slave window size is fixed at 64KB and is not a parameter. An access that decodes outside the map completes locally with PSLVERR rather than stalling the master.

Usage Example

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (32),
    .BASE_ADDR  (32'h8000_0000)
) u_xbar (
    .pclk       (apb_clk),
    .presetn    (apb_rst_n),

    // Master 0 interface
    .m0_apb_PSEL    (m0_psel),
    .m0_apb_PENABLE (m0_penable),
    .m0_apb_PADDR   (m0_paddr),
    .m0_apb_PWRITE  (m0_pwrite),
    .m0_apb_PWDATA  (m0_pwdata),
    .m0_apb_PSTRB   (m0_pstrb),
    .m0_apb_PPROT   (m0_pprot),
    .m0_apb_PRDATA  (m0_prdata),
    .m0_apb_PSLVERR (m0_pslverr),
    .m0_apb_PREADY  (m0_pready),

    // Master 1 interface
    .m1_apb_PSEL    (m1_psel),
    // ... (similar connections)

    // Slave 0-3 interfaces
    .s0_apb_PSEL    (s0_psel),
    // ... (similar connections)
);
```

Testing

All generated crossbars have corresponding test files in `dv/tests/`:

- `test_apbx_xbar_1to1.py` - 100+ transactions, variable delay profiles
- `test_apbx_xbar_2to1.py` - 130+ transactions, arbitration stress tests
- `test_apbx_xbar_1to4.py` - 200+ transactions, address decode validation
- `test_apbx_xbar_2to4.py` - 350+ transactions, full crossbar stress
- `test_apbx_xbar_2to2_mixed.py` - all four version pairings on the generated mixed fabric: sideband value where both ends are APB5, no leak anywhere else, plus a structural check that APB4 ports carry no sideband pins

The mixed test drives its DUT directly; the four legacy tests drive a `rtl/wrappers/*_wrap.sv` scaffold. Those wrappers are hand-written testbench scaffolding, not generator output, so a variant gets one only when a test needs it.

Run tests:

```
pytest
projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/test_apbx
_xbar_2to4.py -v
pytest projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/ -
v  # All variants
```

Design Notes

Arbitration Strategy

- **Round-robin per slave**: Master priority rotates (M0→M1→M0...)
- **Grant persistence**: Once granted, master owns slave until transaction completes
- **Fairness**: No master can starve another master

Address Decoding

- **Parallel decode**: All masters decode addresses simultaneously
- **Registered routing**: Slave selection registered at command acceptance
- **Response routing**: Based on registered slave selection

Timing

- **Back-to-back transactions**: supported with no master-side idle cycles; they do not overlap inside the fabric. 10 pclk cycles per transfer (PREADY-to-PREADY) on `1to1`/`1to4`; **11 on the arbitrated variants** `2to1`, `2to4` and `2to2_mixed`
- **Single-cycle arbitration**: on multi-master variants, new grants are issued the cycle AFTER the previous completion (the arbiter's grant is registered). `1to1` and `1to4` have no arbiter.
- **No datapath pipelining across transactions**: `apb4_slave` is a one-command-at-a-time FSM (IDLE->BUSY->WAIT) that captures the next command only after the previous response is consumed, and the grant is held to the response handshake (`WAIT_GNT_ACK(1)`), so a second command never queues behind an executing one. Sustained cadence is 10 pclk cycles PREADY-to-PREADY against a 9-cycle SETUP-to-PREADY single transfer with one master, and 11 against 10 when arbitrated — in each case one MORE than the latency, because the next transfer's mandatory SETUP cycle cannot overlap the previous transfer's ACCESS. The generator emits no arbiter at all when M = 1, which is the whole difference between the two classes (HAS 5.2).

Known Limitations

1. **Fixed address map**: 64KB regions per slave
   - Can be changed by modifying generator's `addr_offset` calculation
2. **No slave disable**: All slaves always active
   - Could add enable parameter if needed
3. **No timeout handling**: Assumes slaves always respond
   - Add watchdog if needed for unreliable slaves

Generating Custom Variants

For variants beyond 16x16, modify generator limits in `apbx_xbar_generator.py`:

```
if M < 1 or M > 16:  # Change 16 to desired max
    raise ValueError(f"Number of masters must be 1-16, got {M}")
```

Files

- `generate_xbars.py` - Convenience script for generation
- `apbx_xbar_1to1.sv` - 1-to-1 passthrough
- `apbx_xbar_2to1.sv` - 2-to-1 with arbitration
- `apbx_xbar_1to4.sv` - 1-to-4 with address decode
- `apbx_xbar_2to4.sv` - 2-to-4 full crossbar
- `README.md` - This file

References

- APB Specification: ARM IHI 0024C (AMBA APB Protocol v2.0), and IHI 0024E for the APB5 sideband signals
- Documentation: docs/markdown/rtl-amba/apbx/
- Generator: `projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/apbx_xbar_generator.py`
- Base modules: `rtl/amba/apb4/apb4_{slave,master}.sv`, `rtl/amba/apb5/apb5_{slave,master}.sv`
- Formal: `formal/apbx_xbar/` — four all-APB4 harnesses plus `apbx_xbar_2to2_mixed` for the mixed configuration
- Tests: `dv/tests/test_apbx_xbar_*.py`

Not implemented

- **End-to-end APB5 parity.** Every variant's boundary IP deconstructs each transfer into cmd/rsp, so parity terminates there and the IP is instantiated with `ENABLE_PARITY=0`. A variant built with `enable_parity` CHECKS parity at the boundary and REGENERATES it on the far side; the cmd/rsp fabric between the two is outside the protected domain, which is why each port exposes its own `parity_error_*` output. The pins are the IHI 0024E chapter 5 check signals, with odd parity and one bit per 8: `PADDRCHK`, `PCTRLCHK`, `PSELCHK`, `PENABLECHK`, `PWDATACHK`, `PSTRBCHK`, `PAUSERCHK`, `PWUSERCHK`, `PWAKEUPCHK`, `PRDATACHK`, `PREADYCHK`, `PSLVERRCHK`, `PRUSERCHK` and `PBUSERCHK`. No shipped variant enables them; `test_apbx_xbar_parity.py` generates one that does and checks it.

- **Formal proof of version gating (RESTORED 2026-08-29).** The four all-APB4 harnesses (`apbx_xbar_{1to1,2to1,1to4,2to4}`) are joined by `formal/apbx_xbar/apbx_xbar_2to2_mixed/`, which proves the mixed configuration. It replaces the proof lost when the thin core was deleted on 2026-08-27, but it does NOT restate that proof: the thin core took versions as parameters, so every port had every sideband pin and "an APB4 port never sees sideband" was the real content. Here the versions are structural — m0 and s1 have no sideband pins to see anything with — so those properties would pass vacuously and are deliberately omitted. The port-list fact is checked structurally by `test_apbx_xbar_2to2_mixed.py`'s `hasattr` sweep instead.

  What is proven is the part structure does not give for free: the fabric never injects sideband at the APB5 slave on the APB4 master's behalf, and never fabricates completer sideband no APB5 slave produced. On `PWAKEUP`, it proves three things: the APB5 master's wake always reaches the APB5 slave, the slave is never selected without `PWAKEUP`, and the wake is not fabricated from reset. This replaced `ap_pwakeup_dropped`, which pinned the old, reversed direction (issue #102 item 7). Ten cover statements show that sideband and wake really flow, so none of it passes vacuously.

Generated by RTL Design Sherpa | **Last Updated:** 2026-08-13